

STATE SPACE MODELS FOR AUTOMATIC SPEECH TRANSCRIPTION

ROBERT ALEXANDER HANSSEN
Student ID: 197312

SUPERVISOR
Morten Goodwin

University of Agder, 2025
Faculty of Engineering and Science
Department of Engineering and Sciences

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja
2.	Vi erklærer videre at denne besvarelsen: • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høyskoler i Norge, jf. Universitets- og høyskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høyskolens retningslinjer for behandling av saker om fusk.	Ja
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).

Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Nei
Er oppgaven båndlagt (konfidensiell)?	Nei
Er oppgaven unntatt offentlighet?	Nei

Acknowledgements

I would like to thank my supervisor, Morten Goodwin, for his guidance and feedback throughout this project. His insights helped shape the research direction and improve the quality of this work.

I am grateful to the University of Agder for providing access to computational resources that made the experiments possible.

Abstract

Automatic Speech Recognition (ASR) systems must efficiently process long acoustic sequences spanning multiple temporal scales. While Transformer-based architectures have achieved state-of-the-art performance through global self-attention mechanisms, their quadratic computational complexity with respect to sequence length creates a fundamental bottleneck for long-form audio processing. Recently, State Space Models (SSMs) have emerged as a promising alternative, offering linear-time complexity through efficient recurrent formulations. However, standard SSM implementations typically initialize all layers with uniform temporal dynamics, failing to leverage the intrinsic hierarchical structure of speech.

In this work, I investigate the impact of Hierarchical initialization of the SSM encoder layers for ASR. Unlike prior approaches that rely on uniform initialization, this architecture initializes layers with progressively slower temporal decay rates, explicitly encoding an inductive bias that mirrors human auditory processing. Early layers are initialized with faster-decaying states to emphasize short-term acoustic cues, while deeper layers employ slower-decaying states that retain information over longer linguistic timescales. Additionally, I investigate a gating mechanism that learns to apply a sigmoid gate to the SSM branch before the residual addition in each layer, providing enhanced selectivity. To isolate the impact of these architectural choices, I utilize a Simplified Selective SSM encoder.

Systematic ablation studies demonstrate that combining hierarchical initialization with gating consistently improves performance across model configurations. The best configuration, a 30-layer encoder with hierarchical initialization and gating, achieves a Word Error Rate (WER) of 24.45% on test-clean. Experiments across varying depths reveal that hierarchical initialization provides consistent improvements, gating mechanisms enhance selectivity, and their combination yields the best performance at each depth tested. These results suggest that explicit temporal inductive biases are beneficial for SSM-based speech recognition. Notably, the relative error reduction diminishes at greater depths (from 3.4% at 9 layers to 1.6% at 30 layers). This trend suggests that the reduced model width at 30 layers ($D = 200$) may be approaching a representational bottleneck, or that deeper networks are better able to learn temporal structures during optimization.

The absolute WER remains higher than state-of-the-art systems (1.17 to 2.5 percent on standard test sets), and this gap reflects the fundamental scale differences. Model size of approximately 8.6 million parameters trained on 100 hours compared to 80 to 100 million parameters trained on 960 to 10,000 hours for competitive systems. This work establishes hierarchical state dynamics as a promising architectural principle for SSM-based ASR that warrants investigation at larger scales.

Source code available at https://github.com/Roberh18/AI_FALL_2025/tree/main/IKT464.

Contents

Acknowledgements	ii
Abstract	ii
List of Figures	iv
List of Tables	v
1 Introduction	1
1.1 The Problem Space	1
1.2 State Space Models for Automatic Speech Recognition	1
1.3 Research Question and Hypotheses	2
1.4 Scope and Limitations	2
1.5 Summary of Contributions	3
1.6 Report Structure	3
2 Background and Preliminaries	4
2.1 Hierarchical Temporal Processing	4
2.2 Connectionist Temporal Classification (CTC)	5
2.3 End-to-End ASR	6
2.4 State Space Models	6
2.4.1 Continuous-Time Foundations	6
2.4.2 The Evolution to Modern SSMs	7
2.5 Data Processing and Augmentation	8
2.5.1 LibriSpeech	8
2.5.2 Feature Extraction and Subsampling	8
2.5.3 Depthwise Convolution	9
2.6 Training and Evaluation	9
2.6.1 Optimization and Scheduling	9
2.6.2 Mixed Precision Training	9
2.6.3 Evaluation Metric (WER)	9
2.6.4 Decoding	9
3 Related Work	10
3.1 From S4 to Mamba	10
3.2 Hierarchical Approaches	10
3.2.1 Parallel Multi-Scale Processing (Multi-Head SSM)	11
3.2.2 Sequential Multi-Resolution Processing (HiSS)	11
3.2.3 Layer-Wise Hierarchical State Dynamics	11
3.3 Summary of Related Work	12

4	Methodology	13
4.1	System Overview	13
4.2	Acoustic Feature Processing	14
4.2.1	Dataset and Features	14
4.2.2	Spectrogram Computation	15
4.2.3	Data Augmentation	15
4.2.4	Convolutional Subsampling Implementation	15
4.3	Simplified Selective SSM Encoder Architecture	15
4.3.1	Layer Architecture	16
4.3.2	Selective SSM Recurrence	16
4.3.3	Hierarchical State Dynamics	17
4.3.4	Gating Mechanism	18
4.3.5	Initialization Schedule	19
4.3.6	Hierarchical Warmup	20
4.4	Training Procedure	20
4.4.1	Loss Function	20
4.4.2	CTC-Only Architecture	20
4.4.3	Optimization	21
4.4.4	Learning Rate Schedule	21
4.4.5	Mixed Precision Training	22
4.4.6	Decoding	22
4.4.7	Evaluation Metric	22
4.4.8	Implementation Details	22
4.5	Use of Generative AI and Digital Tools	23
4.6	Summary of Methodology	23
5	Experiments and Results	24
5.1	Initial Exploration	24
5.2	Architecture Ablation Study	24
5.3	Depth vs. Width Analysis	25
5.4	Detailed Analysis	26
5.4.1	State Dynamics Visualization	26
5.4.2	Stability and Convergence	27
5.4.3	Comparison with State-of-the-Art	29
5.4.4	Limitations of Analysis	30
5.5	Hypothesis Evaluation	30
5.6	Summary of Key Findings	31
6	Discussion	32
6.1	Interpretation of Results	32
6.2	Comparison with Alternative Approaches	33
6.3	Limitations	33
7	Conclusions	34
8	Future Work	35
	Bibliography	36

List of Figures

2.1	Block diagram of a Continuous-Time SSM	7
2.2	Convolutional subsampling	8
4.1	Architecture of the S-SSSM-CTC model	13
4.2	Hierarchical initialization schedules	20
5.1	Validation WER Evolution Across Model Configurations	26
5.2	Learned Hierarchical Dynamics Parameters ($L = 24$)	27
5.3	Learned Hierarchical Dynamics Parameters ($L = 30$)	27
5.4	Training progression for the encoder	29

List of Tables

2.1	Speech processing hierarchy	4
2.2	Overview of LibriSpeech corpus partitions	8
4.1	Hyper-parameters and Experimental Configuration	21
5.1	Initial exploration on smaller subset of data	24
5.2	Architecture ablation study on LibriSpeech	25
5.3	Impact of dimensionality (Depth vs. Width) on WER	25
5.4	Stability analysis	29
5.5	State-of-the-art ASR results on LibriSpeech.	29
5.6	Training regime comparison.	30

Chapter 1

Introduction

1.1 The Problem Space

Speech transcription, the task of converting spoken language into text is a fundamental component of modern human-computer interaction. It underpins applications such as voice assistants, captioning, meeting transcription, and spoken document retrieval. In machine learning literature, this task is typically referred to as Automatic Speech Recognition (ASR). From a modeling perspective, speech is challenging because it requires modeling of sequences which are both long and structured, where relevant cues span tens to hundreds of milliseconds and align imperfectly. Human speech is an intrinsically hierarchical signal, where phonemic transitions occur within tens of milliseconds, syllables and words span hundreds of milliseconds, while semantic context and sentence-level dependencies require integrating information over several seconds [3, 15, 16].

Transformer-based architectures have achieved State-of-the-Art performance by capturing global interactions via self-attention, but they suffer from quadratic computational complexity with respect to sequence length. For typical speech at 16kHz with 10ms frame shifts, a 20-second utterance produces 2,000 frames, requiring computation over a $2,000 \times 2,000$ attention matrix. This creates a significant bottleneck when processing long-form audio and limits efficiency in resource constrained environments. Conversely, classical recurrent models scale linearly but struggle with vanishing and exploding gradient problems, restricting their ability to retain information over long sequences [1, 32].

While prior work has explored general initialization strategies [13, 11] and hierarchical modeling architectures [9, 2], my review of the literature did not identify prior work that explicitly designs layer-wise initialization schedules for State Space Model (SSM) dynamics to align with the hierarchical timescales of speech processing.

1.2 State Space Models for Automatic Speech Recognition

SSMs offer a promising middle ground for ASR by combining the efficiency of Recurrent Neural Networks (RNNs) with the modeling capabilities of Transformers. They retain a recurrent structure that naturally models sequences but can be implemented with linear-time sequence operations, providing explicit control over how information is stored and forgotten over time. This combination of efficient long-context modeling and flexible temporal memory makes SSMs a natural candidate for speech, where signals span multiple time scales from acoustics to sentence-level semantics. The S4 model demonstrated that linear time-invariant (LTI) SSMs can be parameterized and initialized so that their recurrent dynamics can be implemented as efficient convolutions during training while retaining a recurrent form at inference time [12, 31]. Mamba extended this by introducing selectivity, making state dynamics input-dependent and achieving the context-aware reasoning of attention with the

linear complexity of recurrence [11]. Within the ASR domain, structured variants like S4 and selective variants like Samba-ASR have emerged as strong alternatives to Transformer-based architectures [12, 30].

However, the question of how to best design SSM-native encoder architectures for speech transcription remains open. The current SSM-based ASR approaches often treat the encoder as a homogeneous stack of layers and typically initialize all layers with uniform temporal dynamics, relying entirely on the optimization process to “discover” the hierarchical nature of speech. While some research has explored integrating SSMs into existing architectures by replacing attention mechanisms [25], the idea of explicitly designing the internal state capacity to match the speech processing hierarchy remains underexplored. Current models do not initialize layers with progressively slower temporal decay, nor do they typically employ specialized gating mechanisms [30, 9]. Therefore, this work explores whether using specialized gating mechanisms and varied state dynamics across layers through hierarchical initialization can improve performance over uniform baselines.

1.3 Research Question and Hypotheses

To what extent can explicit temporal inductive biases and gating mechanisms improve the convergence and accuracy of SSMs in ASR tasks?

To address this overarching question, I evaluate the following hypotheses:

1. **The Hierarchy Hypothesis:** Does initializing layers with progressively longer memory spans (hierarchical decay) outperform the standard uniform initialization?

I hypothesize that the impact of hierarchical initialization will scale with network depth, as deeper models have more layers to differentiate and explicitly structure temporal information.

2. **The Gating Hypothesis:** Does the addition of gating mechanisms improve stability, convergence, and accuracy compared to the baseline model?

I hypothesize that gating will provide additional benefits by allowing the model to adaptively filter the input, enabling within-layer selectivity. The gating mechanism enables the model to suppress uninformative regions while emphasizing critical transitions and phoneme boundaries where linguistic information is concentrated.

1.4 Scope and Limitations

This research focuses on the architectural inductive biases of SSMs, prioritizing a rigorous evaluation of hierarchical state dynamics over competition with state-of-the-art models using attention-based decoders and external language model fusion [20, 30, 34]. The study is limited to English speech transcription using the LibriSpeech 100-hour clean benchmark [27]. LibriSpeech was selected because it is a large, well-understood public benchmark that provides a reliable basis for comparison with other published work.

Experiments are conducted at an academic scale (≈ 8.6 million parameters) to ensure reproducibility and computational feasibility. By training an encoder-only Simplified Selective SSM (S-SSSM) model using the Connectionist Temporal Classification (CTC) criterion, this work intentionally isolates the encoder’s performance from the influence of complex decoders.

1.5 Summary of Contributions

This work investigates whether explicit temporal inductive biases and within-layer selectivity can improve SSM encoders for ASR under controlled, academic-scale settings. First, I propose a layer-wise initialization schedule for the SSM state dynamics that assigns progressively longer memory to deeper layers. The schedule is designed to reflect the multi-scale structure of speech, where early layers emphasize short-timescale acoustic variation while deeper layers retain information over longer linguistic spans. Importantly, this modification changes only the initial parameter values and introduces no additional parameters or computational overhead compared to a uniform baseline.

Second, I introduce an explicit gating mechanism inside the S-SSSM layer to increase within-layer selectivity. A sigmoid gate modulates the SSM branch output before the residual connection, allowing the network to suppress uninformative regions and emphasize transitions such as phoneme boundaries. While gating is a standard idea in sequence modeling, its role here is evaluated specifically in combination with hierarchical state initialization within an SSM-native encoder.

Third, to isolate these architectural questions, I implement a S-SSSM with bounded recurrent updates by applying the hyperbolic tangent ($\tanh$) nonlinearity to constrain the recurrent coefficient to $(-1, 1)$. This yields a simple and reproducible experimental framework for studying how initialization and gating affect convergence and recognition accuracy in an encoder-only model trained with the CTC criterion.

Fourth, I provide a systematic empirical evaluation on LibriSpeech 100h through fixed-parameter ablations across depth and width. The best configuration, a 30-layer encoder with hierarchical initialization and gating, achieves a Word Error Rate (WER) of 24.45% on test-clean (and 52.42% on test-other). Across all tested depths, hierarchical initialization yields consistent improvements, while gating provides complementary gains that combine approximately additively. This combination achieves relative error reductions ranging from 1.3% to 3.4% over the uniform baseline at a comparable parameter count.

1.6 Report Structure

The remainder of this report is structured as follows. Chapter 2 provides the necessary background on hierarchical temporal processing in human speech perception, the theoretical foundations of the utilized loss function, and the mathematical formulation of SSMs. Chapter 3 reviews related work, tracing the development from S4 models to modern selective SSMs and discussing existing hierarchical approaches in sequence modeling. Chapter 4 presents the proposed methodology, including the S-SSSM architecture, the hierarchical state-dynamics initialization, and the training setup. Chapter 5 reports the experimental results, including baseline comparisons and controlled ablations of hierarchy and gating, as well as an analysis of depth-width trade-offs. Chapter 6 interprets these findings and discusses the broader implications and limitations, analyzing why specific architectural choices yielded improvements. Finally, Chapter 7 provides the main conclusions for this work, and Chapter 8 outlines potential directions for future work.

Chapter 2

Background and Preliminaries

This chapter establishes the foundational concepts, architectures, and established methods often used in the field of ASR and SSMs. It begins by examining the basis for exploration into the hierarchical temporal processing in speech, followed by the specific loss function CTC, the architectural context of End-to-End ASR, and concludes with the theoretical foundation of SSMs and the key data processing techniques.

2.1 Hierarchical Temporal Processing

Neuroscience research has demonstrated that the human auditory system processes speech through a hierarchy of temporal integration windows. This organization ranges from milliseconds in the primary auditory cortex to seconds in higher-order regions [15, 18]. Specifically, early sensory areas exhibit short integration windows (tens of milliseconds) suitable for extracting phonetic features. In contrast, higher cortical regions integrate information over windows extending to seconds or even minutes [15]. This biological evidence suggests that speech perception is not a homogeneous process but a multi-scale composition. Phonemes combine into syllables, syllables into words, and words into semantic phrases [3].

Despite this well-documented biological hierarchy, existing SSM architectures for ASR often employ homogeneous initialization strategies across all layers [9, 13]. While some multi-head approaches capture timescales in parallel [9], the explicit encoding of speech’s temporal hierarchy vertically across layer depth remains underexplored.

This biological evidence motivates the central hypothesis of this work, which is that deeper layers should ideally possess longer "memory" than shallow layers. The early layers should employ fast-decaying state dynamics suitable for acoustic features ($\tau \approx 10$ to 50 ms), while deeper layers should maintain slower-decaying states appropriate for longer-range linguistic dependencies ($\tau \approx 50$ ms to 2 s). Table 2.1 summarizes this correspondence.

Table 2.1: Correspondence between speech processing hierarchy and SSM layer initialization.

Level	Timescale	Layers	Init (a_0)	Dynamics
Acoustic	10 - 50 ms	$\left[0, \frac{L}{3}\right]$	0.15	Fast Decay (High Plasticity)
Phonetic	50 - 200 ms	$\left[\frac{L}{3}, \frac{2L}{3}\right]$	0.40 - 0.60	Medium Decay
Linguistic	200 ms - 2 s	$\left[\frac{2L}{3}, L\right]$	0.85	Slow Decay (High Stability)

2.2 Connectionist Temporal Classification (CTC)

ASR presents a unique problem where the input length (audio frames) is significantly longer than the output length (text characters), and the alignment between them is unknown. CTC solves this by introducing a special token, the "blank" token ($\emptyset$), and defining a many-to-one mapping $\mathcal{B}$ that collapses repeated characters and removes blanks. Consecutive identical non-blank predictions are merged into a single character and all blank predictions are removed. This makes CTC an alignment-free loss function that enables training of sequence-to-sequence models without frame-level annotations [10]. For ASR, this allows the model to be trained on (audio, text) pairs without needing an explicit, frame-by-frame alignment.

The CTC Mechanism

The CTC framework allows the network to make a prediction for every audio frame. To transform this frame-wise prediction sequence (known as a "path," denoted π) into the final, condensed text sequence, a collapsing function, $\mathcal{B}$, is applied. This function first removes all blank tokens ($\emptyset$) and subsequently collapses any consecutive, identical non-blank characters into a single character.

Example: Consider the target word "hello".

- $\mathcal{B}(\text{h-ee-l-l-o}) = \text{helo}$ (repeated 'e's collapse, adjacent 'l's collapse)
- $\mathcal{B}(\text{h-e-l-}\emptyset\text{-l-o}) = \text{hello}$ (blank separates the two 'l's, preventing collapse)
- $\mathcal{B}(\emptyset\text{-h-}\emptyset\text{-e-l-l-o-}\emptyset) = \text{helo}$ (blanks removed, adjacent 'l's collapse)

The blank token allows the model to output repeated characters in the target sequence by placing a blank token between them in the alignment path.

CTC Loss Function

The goal of the model is to maximize the total probability of the correct output labeling Y given the input features X . Since there are many possible frame-wise paths π that can map to the same target labeling Y (e.g., different durations of the letter 'e'), the CTC loss sums the probabilities of *all* valid paths:

$$P(Y|X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi|X) \quad (2.1)$$

In Equation 2.1, X represents the input acoustic features and Y is the target transcription. The term $\mathcal{B}^{-1}(Y)$ denotes the set of all possible CTC paths π that collapse to the target sequence Y under the collapsing function $\mathcal{B}$. Assuming conditional independence at each step, the probability of a specific path $P(\pi|X)$ is the product of the probabilities of its tokens at each timestep, which is defined in Equation 2.2:

$$P(\pi|X) = \prod_{t=1}^T y_{\pi_t}^t \quad (2.2)$$

Here, T is the total number of timesteps (frames) in the model's output sequence. t is an index iterating over these timesteps. The term $y_{\pi_t}^t$ represents the probability assigned by the model to the token π_t (which includes characters, spaces, and the blank symbol $\emptyset$) at that specific timestep t .

The final CTC loss function ($\mathcal{L}_{CTC}$) is simply the negative logarithm of the total probability $P(Y|X)$ of all valid paths π that map to the target sequence, as presented in Equation 2.3:

$$\mathcal{L}_{CTC} = -\ln P(Y|X) \quad (2.3)$$

The model's only job is then to make this total sum of all correct paths as high as possible. It doesn't have to learn one "perfect" alignment, it just has to learn to output any of the correct alignment paths. While summing thousands of paths may sound like a slow process, the loss can be calculated very efficiently using a dynamic programming algorithm, which finds the total sum without having to check every single path [10].

2.3 End-to-End ASR

In early ASR systems, speech was processed in multiple stages. An acoustic model converted audio to phonemes, a pronunciation dictionary mapped phonemes to words, and a language model predicted word sequences. Each component required separate training and tuning. Modern "end-to-end" systems simplify this by using a single neural network to map audio features directly to text, eliminating the need for separate modules. Architectures like Listen, Attend, and Spell (LAS), use an encoder to extract acoustic features and an attention-based decoder to generate text [4]. Subsequent architectures like Conformer combined self-attention with local convolution to capture both long-range and short range speech structure at scale [14]. This concept was extended by models like ConMamba, which replaced the self-attention module entirely with a selective SSM (Mamba) [25].

2.4 State Space Models

The SSM is a class of sequence models inspired by classical control theory that has emerged as a compelling alternative to Transformers and RNNs for sequence modeling. SSMs combine the distinct advantages of previous architectures, offering the linear inference scaling characteristic of RNNs alongside the parallel training capabilities of Convolutional Neural Networks (CNNs).

2.4.1 Continuous-Time Foundations

Conceptually, an SSM maps a 1-D input signal $x(t)$ to an output $y(t)$ through a latent memory state $h(t)$. In continuous time, this LTI system is defined by a set of linear differential equations:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t) \quad (\text{State Evolution}) \quad (2.4)$$

$$y(t) = \mathbf{C}h(t) + \mathbf{D}x(t) \quad (\text{Output Projection}) \quad (2.5)$$

Figure 2.1 shows how these components interact. The input signal is $x(t)$, and it enters through matrix $\mathbf{B}$, which maps it into the state space. The state transition matrix $\mathbf{A}$ creates a recurrent feedback loop that determines how information persists or decays over time. The current state $h(t)$ is then projected to the output through matrix $\mathbf{C}$, while matrix $\mathbf{D}$ provides a direct path from input to output that bypasses the state dynamics entirely.

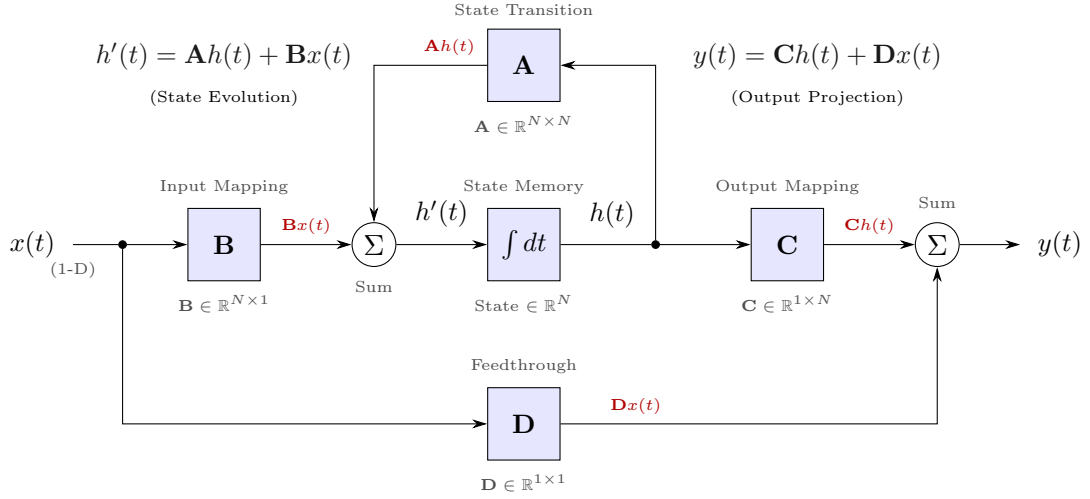


Figure 2.1: Schematic representation of the SSM. The input $x(t)$ drives the latent state $h(t)$ through the input mapping $\mathbf{B}$, while matrix $\mathbf{A}$ governs the state transition dynamics (Equation 2.4). The output $y(t)$ is computed via the output mapping $\mathbf{C}$ and feedthrough path $\mathbf{D}$ (Equation 2.5). The integration symbol ($\int$) represents the continuous-time state evolution from $h'(t)$ to $h(t)$.

The state transition matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the most critical component, as it governs the system’s dynamics by determining how the latent memory decays or persists over time. The input matrix $\mathbf{B} \in \mathbb{R}^{N \times 1}$ maps the scalar input $x(t)$ to the N -dimensional state space, while the output matrix $\mathbf{C} \in \mathbb{R}^{1 \times N}$ projects the internal state back to a scalar output. The feedthrough matrix $\mathbf{D} \in \mathbb{R}^{1 \times 1}$ allows for a direct connection from input to output, independent of the state dynamics.

2.4.2 The Evolution to Modern SSMs

Classical SSMs historically faced a major computational hurdle. Computing the latent state $h(t)$ sequentially was a slow process for long sequences. Modern approaches, such as the Structured State Space (S4) model [12] and Mamba [11], introduced two key innovations to resolve this.

First, they utilize the Zero-Order Hold (ZOH) method to transform the continuous parameters $(\mathbf{A}, \mathbf{B})$ into discrete parameters $(\bar{\mathbf{A}}, \bar{\mathbf{B}})$. For LTI systems, this discretization allows the entire sequence operation to be rewritten as a large convolution filter, enabling massive parallelization on GPUs during training [12, 32]. Additionally, S4 introduced the HiPPO initialization framework to properly parameterize the state matrices, ensuring effective retention of long-range dependencies [12].

Second, while S4 models are LTI (meaning matrices $\mathbf{A}$ and $\mathbf{B}$ are fixed), Mamba introduced the concept of Selectivity. This mechanism makes the parameters into functions of the input x_t , allowing the model to selectively store relevant information and ignore noise. This capability mimics the gating mechanisms found in Long Short-Term Memory (LSTM) models, but is derived from principled SSM foundations [11, 17]. However, making parameters input-dependent $(\mathbf{B}(x_t), \mathbf{C}(x_t), \Delta(x_t))$ breaks the LTI property and prevents the use of efficient convolution-based training. Mamba resolves this through a hardware-aware selective scan algorithm that maintains linear-time complexity $\mathcal{O}(L)$ by optimizing memory access patterns and reducing IO overhead [11, 32].

2.5 Data Processing and Augmentation

2.5.1 LibriSpeech

LibriSpeech is a widely used 960-hour corpus of read English speech derived from LibriVox audiobooks [27]. It is sampled at 16kHz and carefully segmented into training, validation, and test splits. A detailed overview of the entire corpus is provided in Table 2.2.

Table 2.2: Overview of the full LibriSpeech corpus partitions [27].

Split Type	Subset	Utterances	Hours	Speakers
Training	train-clean-100	28,539	~ 100.6	251
	train-clean-360	104,014	~ 363.6	921
	train-other-500	148,688	~ 496.7	1,166
Validation	dev-clean	2,703	~ 5.4	40
	dev-other	2,864	~ 5.3	33
Testing	test-clean	2,620	~ 5.4	40
	test-other	2,939	~ 5.1	33

The dataset used in this work was sourced from the Hugging Face Hub [26].

Using this standard benchmark ensures that the results can be directly compared to other published work. While state-of-the-art systems often train on the full 960 hours or augment with even larger datasets like GigaSpeech (10,000 hours) [6], the 100-hour subset provides a computationally feasible scale that is sufficiently large to draw meaningful conclusions regarding the architectural modifications proposed.

2.5.2 Feature Extraction and Subsampling

Raw audio is first converted into 80-dimensional log-Mel spectrograms. Then, to reduce the computational load for the subsequent SSM Encoder, a Convolutional Subampler is applied, reducing the input sequence length T by a factor of $4\times$. This two-layer process, illustrated in Figure 2.2, is standard in modern ASR architectures and uses stacked 1D convolutional layers with strides greater than one. This technique maps the 80-dimensional acoustic features into the model’s hidden dimension space while drastically lowering the sequence length and recurrent computational cost [14].

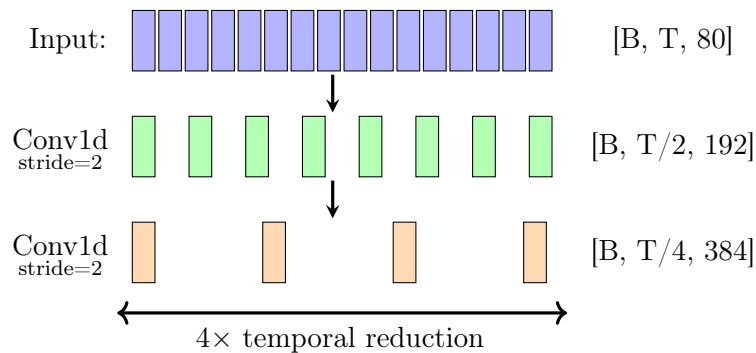


Figure 2.2: Convolutional subsampling reduces sequence length by $4\times$ through two strided convolutions, mapping 80-dimensional log-Mel features to 384-dimensional representations. This reduces computational cost for subsequent encoder layers.

2.5.3 Depthwise Convolution

Within each SSM block, a **depthwise convolution** (dwconv) is applied to the input. Unlike a standard convolution, which mixes information across all channels, a depthwise layer filters each channel independently before the main selective recurrence [14].

$$\tilde{x}_k = \text{DWConv}(x_k) \quad (2.6)$$

Here, x_k represents the input sequence for the k^{th} feature channel, and $\tilde{x}_k$ is the output of the depthwise convolution for that same channel. DWConv denotes a learned 1D kernel that is applied independently to each channel. This approach provides two key advantages. First, it significantly reduces the parameter count and computational complexity compared to a standard convolution. Secondly, it captures local temporal context within each feature channel independently, which is effective in separating spectral features before the selective recurrence mechanism [14].

2.6 Training and Evaluation

2.6.1 Optimization and Scheduling

Sequence models are often optimized using the AdamW optimizer, which is preferred as it decouples the weight decay regularization from the gradient-based updates, often leading to better generalization. This is often paired with a Cosine Learning Rate Schedule with Warmup. This scheduler enables the learning rate to gradually increase during a brief "warmup" period to prevent initial instability, before slowly decaying according to a cosine curve [23].

2.6.2 Mixed Precision Training

To ensure computational feasibility on standard hardware, techniques like Automatic Mixed Precision (AMP) are commonly used [24]. This method performs the majority of calculations using 16-bit floating point (FP16) to reduce memory usage and increase speed, while selectively keeping critical operations (such as weight updates) in 32-bit (FP32) to maintain numerical stability and accuracy [24].

2.6.3 Evaluation Metric (WER)

The standard metric for measuring ASR performance is WER. It measures the minimum number of edit operations (substitutions, deletions, or insertions) required to transform the model's predicted transcription (hypothesis) into the ground-truth text (reference) [21].

The WER is calculated as:

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (2.7)$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of words in the reference text [21].

2.6.4 Decoding

After a model generates frame-by-frame probabilities (logits), a decoding algorithm is required to convert them into a final text sequence. The simplest method is Greedy CTC Decoding, which selects the most probable token at each time step [10]. Finally, the resulting sequence is processed to remove the blank symbol and collapse repeated tokens [10].

Chapter 3

Related Work

SSMs have recently emerged as a competitive alternative to recurrent and Transformer-based architectures for long-context sequence modeling. In contrast to Transformers, which rely on quadratic self-attention, SSMs offer linear-time sequence operations while retaining a recurrent interpretation which is suitable for sequential data such as speech. The S4 architecture demonstrated how a carefully parameterized linear state space can achieve strong performance, while Mamba augmented this framework with content-dependent selectivity [12, 11]. These advancements have made SSMs particularly attractive for applications requiring efficient processing of long sequences.

However, the application of these models to ASR raises questions. First, S4 relies on fixed, globally applied dynamics that do not adapt to the input, which can limit its ability to focus on the most relevant parts of the signal. Second, although Mamba introduces input-dependent dynamics, current implementations typically treat the encoder as a homogeneous stack of layers. Finally, state-of-the-art SSM-based ASR systems often rely on massive data scale, complex attention-based decoders, and language model fusion to hide encoder deficiencies.

In this chapter, I review the progression from S4 to Mamba and recent SSM-based ASR systems (Section 3.1), and then discuss hierarchical approaches that attempt to align model structure with the multi-timescale nature of speech (Section 3.2).

3.1 From S4 to Mamba

The S4 model utilizes techniques derived from control theory, such as the HiPPO framework, to properly initialize and parameterize the core SSM matrices (A, B, C) as structured matrices [12]. This allows the SSM, which is recurrent by nature, to be computed as a highly efficient convolution during training. The Mamba architecture builds upon S4, and its key innovation is the selective scan mechanism [11]. Mamba makes the B , C , and discretization step (Δ) parameters input dependent.

This allows the model to "selectively" decide how much of the current input to focus on versus how much of the past state to retain, giving it the context-aware capabilities of attention while maintaining linear $O(T)$ complexity [11, 32].

3.2 Hierarchical Approaches

Speech is inherently hierarchical, with information spanning timescales from milliseconds (acoustic details) to seconds (sentence-level semantics). A natural way to reflect this structure in a model is to introduce hierarchy in how temporal information is represented and processed. The concept of hierarchy in SSMs has been explored in ways that are related to, but distinct from, the approach taken in this work. In the following subsections, I first

review methods that introduce hierarchy via parallel multi-scale processing (Stateformer), then methods that build multi-resolution stacks (HiSS), and finally contrast these with the layer-wise hierarchical state dynamics used in the proposed encoder.

3.2.1 Parallel Multi-Scale Processing (Multi-Head SSM)

One prominent approach to hierarchy is the Multi-Head SSM (MH-SSM) proposed in the Stateformer architecture [9]. Similar to Multi-Head Attention, this method employs multiple SSM "heads" processing the input in parallel. However, instead of attending to different subspaces, each head is initialized differently to capture diverse temporal dynamics. For example, separate heads with $d_{\text{state}} \in \{16, 32, 64, 128\}$ may simultaneously capture a range of temporal dynamics, both local acoustic transitions and global sentence dynamics. These parallel representations are subsequently concatenated and mixed through a learned projection.

While effective, this approach introduces significant computational redundancy. By processing all timescales at all layers in parallel, the parameter count and memory footprint increase proportionally to the number of heads. Furthermore, this parallel design does not inherently force the model to build high-level abstractions from low-level features, as both "fast" and "slow" heads operate on the same input representation at every depth.

3.2.2 Sequential Multi-Resolution Processing (HiSS)

A more explicit structural hierarchy is proposed in Hierarchical SSMs (HiSS) [2]. Originally designed for continuous sequence-to-sequence modeling of sensor data, this architecture creates a temporal hierarchy by vertically stacking models that operate at different resolutions. A low-level SSM processes the input in short, non-overlapping chunks, typically of size 10. These chunk representations are then projected and fed into a high-level SSM that models the sequence at a coarser temporal resolution.

While effective for sensor data, I argue that this approach presents distinct weaknesses for speech transcription. The reliance on chunking introduces artificial boundaries, and phonemes or words that span across these boundaries may suffer from artifacts or require complex overlapping window schemes to resolve.

Furthermore, this discretization compromises the fundamental advantage of the continuous-time recurrence found in standard SSMs. By forcing the signal into rigid windows, the model sacrifices the fluid, sample-by-sample update capability that makes SSMs particularly well-suited for streaming speech applications.

3.2.3 Layer-Wise Hierarchical State Dynamics

In contrast to Stateformer and HiSS, the encoder implemented in this work maintains a single, efficient stack of SSM layers with a *constant* state dimensionality d_{model} across depth. Rather than compressing the state into progressively smaller dimensions or introducing additional heads, hierarchy is introduced through *layer-wise state initialization*. Each SSM layer is equipped with depth-dependent initial parameters governing its temporal decay and sensitivity. Early layers are initialized with fast-decaying dynamics suited for short-timescale acoustic features, while deep layers are initialized with slow-decaying dynamics suited for long-range linguistic context. This approach adds no parameters and no computational overhead compared to a uniform baseline. The detailed formulation is presented in Section 4.3.3.

3.3 Summary of Related Work

SSMs represent a significant leap forward in efficient sequence modeling, solving the long-range dependency problem that plagued RNNs without the quadratic cost of Transformers. Within ASR, this has enabled SSM-based encoders to emerge as competitive alternatives to attention-heavy architectures. While selective variants like Mamba have solved the selection problem, allowing models to ignore irrelevant data, current SSM-based ASR systems still lack an explicit structural understanding of temporal hierarchy. Existing solutions, such as parallel multi-scale heads or chunked stacking, introduce potentially unnecessary computational complexity or boundary artifacts that may disrupt the continuous nature of speech signals.

There remains a gap in the literature for an SSM-based ASR architecture that inherently respects the hierarchical timescales of speech without abandoning the simplicity of a uniform, flat encoder stack. The following chapter details the methodology used to address this gap, proposing layer-wise initialization of state dynamics to encode temporal hierarchy directly into the architecture.

Chapter 4

Methodology

This chapter details the specific architecture and training methodology of this work. The core components include acoustic feature processing, the (S-SSSM) encoder, the implementation of the novel Hierarchical State Dynamics, and the overall training procedure. The primary goal was to establish a strong, stable baseline model and then systematically evaluate the impact of hierarchical state dynamics and gating mechanisms.

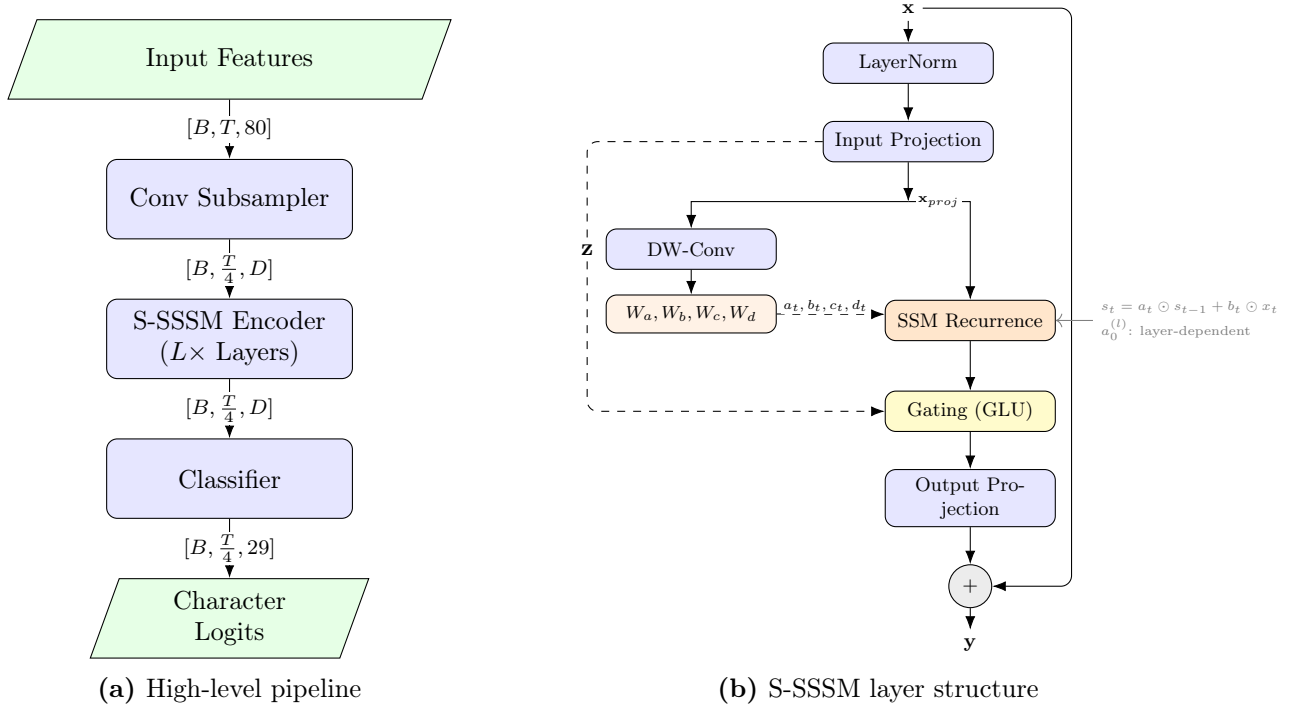


Figure 4.1: Architecture of the S-SSSM-CTC model. (a) The high-level pipeline transforms 80-dimensional log-Mel features into character logits through convolutional subsampling, L stacked S-SSSM layers, and a linear classifier. (b) Each S-SSSM layer applies layer normalization, projects the input (splitting into content and gate branches when gating is enabled), applies depthwise convolution and selective SSM recurrence, then combines with the gate and residual connection. The SSM decay parameter a_0 is initialized according to layer depth to create hierarchical temporal dynamics. Orange highlights the SSM recurrence; yellow highlights the gating mechanism.

4.1 System Overview

The implementation is an encoder-only ASR architecture with CTC [10]. Figure 4.1 illustrates both the complete processing pipeline and the internal layer structure. The system transforms raw audio into character-level transcriptions through three main stages.

Feature Extraction and Subsampling

Raw 16kHz audio waveforms are converted into 80-dimensional log-Mel spectrograms. A convolutional subsampler (Figure 4.1a) then reduces the temporal resolution by a factor of four while projecting the features into the model’s hidden dimension D . This subsampling is essential for computational efficiency, reducing a 20-second utterance from 2000 frames to 500 frames before the encoder.

S-SSSM Encoder

The subsampled features pass through a stack of S-SSSM layers. As shown in Figure 4.1b, each layer applies layer normalization, projects the input (splitting into content and gate branches), applies depthwise convolution for local context, and processes the sequence through the selective SSM recurrence. The key architectural choice investigated in this work is how the SSM dynamics are initialized. Either uniformly across all layers, or hierarchically to match the multi-scale temporal structure of speech.

Classification

A linear projection maps the encoder output to vocabulary logits over $|V| = 29$ tokens (26 letters, space, apostrophe, and the CTC blank). During inference, greedy CTC decoding selects the most probable token at each timestep, then collapses repeated tokens and removes blanks to produce the final transcription.

4.2 Acoustic Feature Processing

The input to the model is a log-Mel spectrogram, a standard acoustic representation for speech recognition that captures both temporal and frequency information in a perceptually meaningful way.

4.2.1 Dataset and Features

Experiments were conducted using a subset of the full LibriSpeech corpus [27]. This corpus provides both “clean” and “other” test sets, allowing for a distinct evaluation of performance on clear speech versus speech with diverse acoustic challenges. This work utilizes the **train-clean-100** subset for training. Validation is performed on **dev-clean**, while the final evaluation leverages both **test-clean** and **test-other** to measure performance across different acoustic conditions. As detailed in Table 2.2, the validation and test sets are disjoint from the training speakers to ensure fair evaluation.

The choice to train on 100 hours rather than the full 960-hour corpus reflects the focus of this work on architectural exploration rather than state-of-the-art performance. Training on the smaller subset allows for rigorous iterative testing of architectural variants that would be computationally prohibitive at full scale. The “clean” subsets consist of high-quality recordings with high signal-to-noise ratios. In contrast, the “other” splits contain more challenging audio with varying accents, microphone qualities and background noise.

The models presented in this work were trained exclusively on the **train-clean-100** subset. While results on the **test-other** dataset are included, this is only for comparison with prior work (which typically utilizes the full 960-hour corpus). This restriction means that evaluation on **test-other** represents an out-of-distribution generalization test, assessing how well performance transfers to more acoustically challenging conditions. Consequently, the performance gap between **test-clean** and **test-other** reflects this distribution shift rather than a fundamental limitation of the architecture

4.2.2 Spectrogram Computation

The input representation follows standard practices in modern ASR systems [14]. Raw 16kHz audio waveforms are converted to 80-dimensional log-Mel spectrograms. The computation applies a 25 ms Hann window (400 samples) with a 10 ms hop length (160 samples), resulting in a 15 ms overlap between frames. The spectral analysis utilizes a 400-point Fast Fourier Transform (FFT), band-limited to a frequency range of 20 Hz to 7600 Hz to capture speech-relevant frequency components. The resulting spectrogram is normalized per-utterance by subtracting the mean and dividing by the standard deviation. This utterance-level normalization improves robustness to variations in recording volume and acoustic conditions.

4.2.3 Data Augmentation

To improve generalization, SpecAugment is applied with a configuration of $m_F = 2$ frequency masks ($F = 15$) and $m_T = 2$ time masks ($T = 50$) [28]. For each mask, the width is randomly sampled up to the maximum value, occluding different frequency bands and time segments in each training example. This augmentation is applied prior to batching, ensuring unique masking patterns for every sample. By obscuring parts of the input, the model learns to rely on broader contextual information rather than specific spectral or temporal features, improving robustness without requiring additional training data.

4.2.4 Convolutional Subsampling Implementation

The acoustic features are passed through a convolutional subsampler (Figures 2.2 & 4.1a). The features are temporally compressed using two strided 1D convolutional layers before entering the SSM encoder. The first convolution maps the 80-dimensional input to $D_{model}/2$ channels with stride 2, and the second convolution maps to D_{model} channels with stride 2, yielding a total temporal reduction factor of 4. Both layers use a kernel size of 5 with ReLU activations. For a model size of ($D_{model} = 336$), the subsampling transforms an input of shape $[B, T, 80]$ to $[B, T/4, 336]$, where B denotes the batch size and T denotes the sequence length. This $4\times$ subsampling reduces the sequence length from T to $T/4$, substantially lowering the computational cost for the subsequent SSM encoder. A 20-second utterance at 16kHz with 10ms frame shift produces 2000 frames, which the subsampler reduces to 500 frames before the SSM encoder. The choice of a $4\times$ reduction factor follows the Conformer architecture [14], which demonstrated this to be an effective trade-off between computational cost and information preservation for speech recognition.

4.3 Simplified Selective SSM Encoder Architecture

The baseline S-SSSM follows an end-to-end encoder-only architecture with CTC loss. The architecture incorporates the selective gating philosophy of Mamba [11], but uses stable bounded recurrences rather than exact discretizations for numerical stability during training. The use of CTC-only decoding, rather than a hybrid CTC-attention architecture [34], serves three purposes.

First, it simplifies the model and reduces the number of hyper-parameters. Second, CTC enforces a monotonic alignment that stabilizes training on limited data [19]. Third, and most importantly for this study, it isolates the encoder’s contribution to performance, enabling cleaner interpretation of the ablation results.

Unlike the standard Mamba architecture, which relies on a learned timescale parameter to discretize the continuous system parameters ($\mathbf{A}, \mathbf{B}$) via the ZOH, this ‘Simplified’ implementation bypasses explicit discretization. Instead, it directly learns the recurrent decay $\mathbf{a}_t$ and input gate $\mathbf{b}_t$, significantly reducing complexity while maintaining the selective properties

essential for performance.

The ‘‘S-SSSM’’ terminology used here refers to the specific implementation used in this work, employing bounded tanh recurrence. This is distinct from the S5 (Simplified State Space) architecture of Smith et al. [31], which uses MIMO dynamics and parallel associative scans.

4.3.1 Layer Architecture

Each S-SSSM layer processes the input sequence through layer normalization, input projection, depthwise convolution, selective recurrence, and output projection with residual connection. Given an input sequence $\mathbf{x}$ of shape $[B, T', D]$, where B is the batch size, $T' = T/4$ is the sub-sampled sequence length, and $D = D_{\text{model}}$ is the hidden dimension, the layer first applies layer normalization.

The normalized input is then projected. When gating is enabled, the projection doubles the dimension and splits the result into a content branch $\mathbf{x}_{\text{proj}}$ and a gate branch $\mathbf{z}$. When gating is disabled, the projection maintains the same dimension. A depthwise convolution with kernel size $k = 5$ then provides local temporal context before the recurrence. The convolution is applied independently to each channel, capturing local temporal patterns without mixing information across feature dimensions. Finally, the layer output is projected back to the model dimension, passed through a dropout layer ($p = 0.1$) for regularization, and added to the original input via a residual connection.

4.3.2 Selective SSM Recurrence

At the core of each layer is the selective recurrence mechanism. The depthwise-convolved features $\tilde{\mathbf{x}}_t$ are used to compute the gates using bounded activations to modulate the flow of information (capturing local context), while the projected features $\mathbf{x}_{\text{proj},t}$ are integrated into the memory state (preserving features).

For each position t , the recurrence first computes four input-dependent gate values using the convolved input $\tilde{\mathbf{x}}_t$:

$$\text{State Decay:} \quad \mathbf{a}_t = \tanh(\mathbf{a}_0 + \mathbf{W}_a \tilde{\mathbf{x}}_t) \quad (4.1)$$

$$\text{Input Gate:} \quad \mathbf{b}_t = \sigma(\mathbf{b}_0 + \mathbf{W}_b \tilde{\mathbf{x}}_t) \quad (4.2)$$

$$\text{Output Gate:} \quad \mathbf{c}_t = \sigma(\mathbf{c}_0 + \mathbf{W}_c \tilde{\mathbf{x}}_t) \quad (4.3)$$

$$\text{Feedthrough Gate:} \quad \mathbf{d}_t = \mathbf{W}_d \tilde{\mathbf{x}}_t + \mathbf{d}_0 \quad (4.4)$$

Here, σ denotes the sigmoid function. $\mathbf{W}_a, \mathbf{W}_b, \mathbf{W}_c, \mathbf{W}_d \in \mathbb{R}^{D \times D}$ are learned weight matrices, and $\mathbf{a}_0, \mathbf{b}_0, \mathbf{c}_0, \mathbf{d}_0 \in \mathbb{R}^D$ are learnable bias vectors. The bounded activations ($\tanh \in [-1, 1]$ and $\sigma \in [0, 1]$) ensure numerical stability by preventing exponential growth in the hidden state.

The hidden state $\mathbf{s}_t \in \mathbb{R}^D$ then evolves by integrating the content branch $\mathbf{x}_{\text{proj},t}$ according to the recurrence:

$$\mathbf{s}_t = \mathbf{a}_t \odot \mathbf{s}_{t-1} + \mathbf{b}_t \odot \mathbf{x}_{\text{proj},t} \quad (4.5)$$

where $\odot$ denotes element-wise multiplication and $\mathbf{s}_0 = \mathbf{0}$. Finally, the layer output is computed by combining the recurrent state and the direct feedthrough path:

$$\mathbf{y}_t = \mathbf{c}_t \odot \mathbf{s}_t + \mathbf{d}_t \odot \mathbf{x}_{\text{proj},t} \quad (4.6)$$

Each parameter controls a specific aspect of information flow. The decay parameter $\mathbf{a}_t$ affects memory retention, where values near ± 1 indicate slow decay (long memory) and values approaching 0 represent fast decay. The input gate $\mathbf{b}_t$ determines the magnitude of new content $\mathbf{x}_{\text{proj},t}$ written into the state, while the output gate $\mathbf{c}_t$ modulates the exposure of

this accumulated state to the subsequent layer. Finally, the feedthrough gate $\mathbf{d}_t$ establishes a direct residual path, allowing the input content to bypass the recurrence entirely.

In standard Mamba, the state transition uses ZOH discretization where $\bar{\mathbf{A}} = \exp(\Delta\mathbf{A})$, with the exponential function naturally ensuring stability while preserving the continuous-time interpretation [11].

The simplified formulation used in this work bounds the decay coefficient directly through tanh instead, which constrains $\mathbf{a}_t \in (-1, 1)$ without requiring explicit discretization. This design choice was made to ensure training stability in the absence of hardware-optimized selective scan kernels, trading continuous-time exactness for experimental robustness. The resulting recurrence behaves similarly to a Gated Recurrent Unit (GRU) [7, 17], but with the SSM-inspired separation of state decay and input gating that enables the hierarchical initialization investigated in this work.

Note that while LTI-SSMs allow for parallel training via convolution, selective variants with input-dependent parameters require specialized parallel scan algorithms for efficient training [11]. The current implementation uses a sequential recurrence for simplicity and numerical transparency, which is sufficient for the architectural investigation in this work but does not leverage the full computational efficiency of optimized selective scan kernels.

4.3.3 Hierarchical State Dynamics

The key contribution of this work is the layer-wise initialization of state dynamics to reflect the hierarchical nature of speech processing. I hypothesize that an encoder where layers share the same state size but differ in their learned decay rates would more effectively capture the multi-scale nature of speech. Rather than using uniform initialization across all layers, each layer’s bias parameters ($\mathbf{a}_0, \mathbf{b}_0, \mathbf{c}_0$) are initialized according to its depth in the encoder. Early layers are initialized with faster-decaying dynamics and higher sensitivity to the current input, encouraging them to focus on short-timescale acoustic variations (10 - 50 ms). Middle layers use intermediate decay rates, capturing phonetic structure over medium timescales (50 - 200 ms). Late layers are initialized with slower-decaying dynamics and lower input sensitivity, allowing them to accumulate and maintain longer-term linguistic context (hundreds of milliseconds to seconds). This is implemented by initializing the recurrent parameters as functions of the normalized layer index $\rho \in [0, 1]$. Each parameter controls a specific aspect of the recurrence:

State decay (a_0): Controls how much of the previous hidden state is retained at each timestep. Early layers receive low values (fast forgetting, short memory), while deep layers receive high values (slow forgetting, long memory).

Input sensitivity (b_0): Controls how strongly new inputs are written into the state. Early layers receive high values (responsive to fine-grained acoustic detail), while deep layers receive low values (more dependent on accumulated context).

Output weight (c_0): Controls how much the hidden state contributes to the layer output, interpolated linearly across depth.

The specific initialization range of $a_0 \in [0.15, 0.85]$ was selected based on the behavior of the tanh activation function applied to the decay parameter. For example, in a 24-layer encoder, layer 0 is initialized with $a_0 = 0.15$ (fast decay) while layer 23 receives $a_0 = 0.85$ (slow decay). After passing through the tanh activation, these values correspond to retention rates of approximately 15% and 69% per timestep, respectively. This creates a functional hierarchy where deep layers retain information roughly 4–5 times longer than early layers.

Importantly, this range keeps the parameters within the steep, linear portion of the tanh curve, avoiding the saturated regions near ± 1 where the curve flattens out. This ensures that gradients can flow efficiently during training. The state vector itself remains in $\mathbb{R}^{d_{\text{model}}}$ in all layers, with no projection bottleneck or reduction in dimensionality. The only difference between configurations is the initial bias values.

These values were selected as a theoretically motivated starting point. Since initial experiments confirmed that this range yielded an immediate performance improvement over the uniform baseline, this configuration was retained for the remainder of the study. A systematic search for the optimal initialization boundaries is reserved for future work.

4.3.4 Gating Mechanism

When gating is enabled, a Gated Linear Unit (GLU) structure provides additional selectivity. The gate branch $\mathbf{z}$, which bypassed the recurrence, forms a sigmoid-activated gate that modulates the SSM output:

$$\mathbf{y}_{\text{gated}} = \mathbf{g} \odot \mathbf{y}_{\text{ssm}} = \sigma(\mathbf{z}) \odot \mathbf{y}_{\text{ssm}} \quad (4.7)$$

This mechanism allows the network to suppress irrelevant features or reset information flow based on the current input context, independent of the accumulated state history. The design is motivated by the observation that speech contains highly variable information density. Consonant releases and phoneme boundaries carry concentrated linguistic information, while steady-state vowels may span hundreds of milliseconds with little change. Gating provides a mechanism to adaptively weight these different regions.

Enhanced Selectivity

The S-SSSM model already provides some selectivity through its input-dependent parameters (a_t, b_t, c_t) . However, these parameters are tightly coupled to the recurrent state update. An alternative approach is to add an explicit gating mechanism that can bypass or modulate the recurrent path entirely. This idea has proven successful in earlier recurrent architectures. The performance of LSTM and GRU can be largely attributed to their learned gates, which control what information flows through the network at each timestep.

Not All Frames Are Equal

Speech contains highly variable information density. Some moments are critical while others are redundant. Consonant releases, vowel onsets, and phoneme boundaries occur in brief temporal windows (5 to 20 ms). These transitions carry most of the linguistic information. By contrast, steady-state vowels or silence can span hundreds of milliseconds with little change. A recurrence that processes all frames identically may miss these distinctions. It might waste capacity on repetitive regions while under-emphasizing the critical transitions [16]. Furthermore, real-world speech is often recorded in noisy conditions which adds another challenge. Recordings often contain background noise, reverberation, or channel distortions. An adaptive mechanism that can suppress corrupted frames or irrelevant patterns would be valuable. Gating provides exactly this capability. It learns to amplify important features and suppress noise based on the input context.

GLUs in SSMs

This work implements gating through a GLU structure. The architecture splits the input into two parallel paths. First, the input projection expands the representation to $2 \times D_{\text{model}}$ dimensions. This expanded representation is then divided into two branches. The content branch ($\mathbf{x}_{\text{proj}}$) follows the standard SSM path. It passes through depthwise convolution and

the SSM recurrence to produce $\mathbf{y}_{\text{ssm}}$ (output of SSM Recurrence block). The gate branch ($\mathbf{z}$) takes a different route. It bypasses the recurrence entirely and instead forms a learned gate through a sigmoid activation ($\mathbf{g} = \sigma(\mathbf{z})$). These two streams are combined through element-wise multiplication. This design allows the network to learn which parts of the recurrent output to pass forward and which to suppress. The decision depends on the current input but remains independent of the accumulated state history. This separation is important because it gives the model a direct path to suppress information without having to manipulate the recurrent dynamics.

Complementarity with Hierarchical Dynamics

The gating mechanism is complementary to hierarchical state initialization. While hierarchical initialization establishes layer-specific temporal biases (fast decay in early layers, slow decay in deep layers), gating provides within-layer selectivity at each timestep. Together, they form a two-level architectural hierarchy. Gating operates within each layer at every timestep. It provides horizontal selectivity by deciding which information to emphasize or suppress frame by frame. The combination creates a two-level control mechanism. Between layers, the hierarchy establishes what timescales each layer should focus on. Within each layer, gating determines which specific inputs deserve attention at each moment.

4.3.5 Initialization Schedule

To implement the proposed hierarchy, initialization are defined with schedules that automatically adjust the recurrent parameters based on the layer’s depth. This creates a gradual transition from early layers, which are tuned for fast-changing acoustic details, to deeper layers designed to retain slower linguistic context.

Let l denote the layer index (0-indexed) and L denote the total number of layers. The normalised layer position ρ is defined as:

$$\rho = \frac{l}{\max(1, L - 1)} \quad (4.8)$$

This maps early layers ($l = 0$) to $\rho = 0$ and late layers ($l = L - 1$) to $\rho = 1$. The decay parameter $\bar{a}^{(l)}$ follows an exponential schedule:

$$\bar{a}^{(l)} = a_{\text{early}} \cdot \left(\frac{a_{\text{late}}}{a_{\text{early}}} \right)^{\rho} \quad (4.9)$$

With $a_{\text{early}} = 0.15$ and $a_{\text{late}} = 0.85$, this yields target decay values of approximately 0.15 for the earliest layer and 0.85 for the deepest layer. After the tanh activation, these correspond to retention rates of approximately 0.15 and 0.69 per timestep respectively. The input sensitivity parameter $\bar{b}^{(l)}$ follows a similar exponential schedule:

$$\bar{b}^{(l)} = b_{\text{early}} \cdot \left(\frac{b_{\text{late}}}{b_{\text{early}}} \right)^{\rho} \quad (4.10)$$

With $b_{\text{early}} = 0.35$ and $b_{\text{late}} = 0.12$, early layers receive higher input sensitivity to respond to fine-grained acoustic detail, while later layers are less sensitive to individual inputs and more dependent on accumulated state.

The output weight $\bar{c}^{(l)}$ follows a linear schedule:

$$\bar{c}^{(l)} = c_{\text{early}} + (c_{\text{late}} - c_{\text{early}}) \cdot \rho \quad (4.11)$$

With $c_{\text{early}} = -0.35$ and $c_{\text{late}} = 0.20$, this produces sigmoid outputs ranging from approximately 0.41 to 0.55.

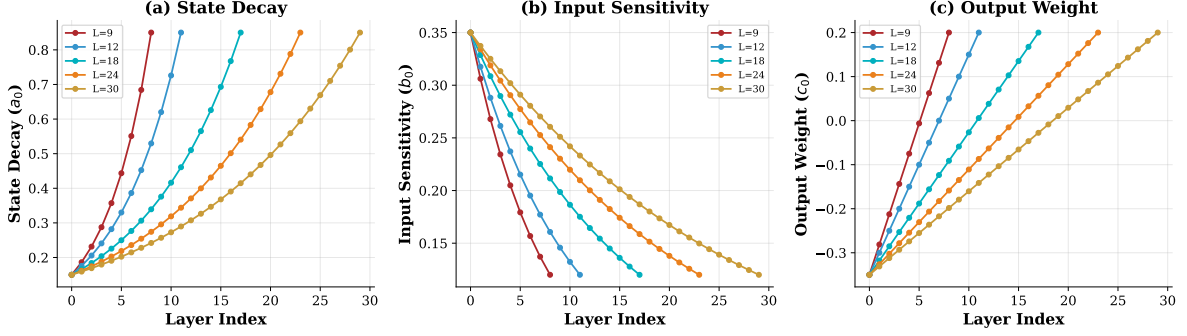


Figure 4.2: Hierarchical initialization schedules for State Decay (a_0), Input Sensitivity (b_0), and Output Weight (c_0) across the varying model depths (L) utilized in the experiments.

Figure 4.2 visualizes these initialization schedules across the different model depths ($L \in \{9, 12, 18, 24, 30\}$) explored in the experimental section. As illustrated, the exponential and linear progressions are preserved regardless of the total layer count, ensuring that every configuration possesses a consistent acoustic-to-linguistic hierarchy.

4.3.6 Hierarchical Warmup

The implementation includes an optional ‘Hierarchy Freeze’ mechanism that can freeze the state dynamics parameters for a specified number of initial epochs. The motivation for this feature was to force the remaining network weights to adapt to the imposed temporal structure before allowing fine-tuning of the dynamics themselves. However, preliminary experiments demonstrated that the model converges effectively without this constraint. Consequently, to simplify the training pipeline, all parameters are trained from the first epoch for the reported results. The freeze mechanism remains available in the implementation for future investigation with different model scales or training regimes where it may prove beneficial.

4.4 Training Procedure

4.4.1 Loss Function

The CTC loss function enables alignment-free training on (audio, text) pairs [10] by introducing a blank token and marginalizing over all valid alignments between the output sequence and the target transcription. The implementation uses the standard PyTorch CTC loss function with `zero_infinity=True` to handle numerical edge cases.

The final linear layer converts the encoder output to vocabulary logits according to equation 4.12, where $|V| = 29$ (26 letters + space + apostrophe + CTC blank token).

$$\mathbf{logits} = \text{Linear}(\text{LayerNorm}(\mathbf{h}_{\text{encoder}})) \in \mathbb{R}^{B \times T' \times |V|} \quad (4.12)$$

4.4.2 CTC-Only Architecture

While attention-based decoders are common in state-of-the-art systems, an encoder-only CTC architecture was selected for three reasons. First, the assumption that current outputs are independent of previous ones simplifies the architecture. While this is a simplification, it makes the model faster and better suited for streaming since it does not need to wait for the full sequence to generate text [19]. Second, CTC enforces a strict left-to-right monotonic alignment, which helps stabilize the training process and is less susceptible to alignment failures in noisy audio compared to attention mechanisms [19, 34]. Third, and most importantly

for this work, using pure CTC isolates the encoder’s contribution to ASR performance, enabling a cleaner ablation study of the proposed hierarchical initialization without the results being influenced or obscured by a complex attention decoder. I recognize that combining CTC and attention often yields the best error rates [34], but pure CTC is the most effective tool for this specific architectural study.

4.4.3 Optimization

Models are optimized using AdamW [22] with the hyper-parameter configuration summarized in Table 4.1. This optimizer decouples weight decay regularization from gradient-based updates, often leading to better generalization.

Specifically, the weight decay value of 0.05 follows the recommendation of Loshchilov and Hutter [22], who found this effective across a range of tasks. The momentum coefficients follow the original Transformer paper [33], which have been widely adopted in sequence modeling. Additionally, gradient clipping threshold of 5.0 is a commonly used value that prevents gradient explosions during training on long sequences.

The model dimension D and number of layers L are varied inversely to maintain a fixed parameter budget of approximately 8.6 million parameters, enabling fair comparisons across different depth-width configurations.

Table 4.1: Hyper-parameters and Experimental Configuration. The model depth (L) and width (D_{model}) are varied inversely to maintain a fixed parameter budget (see Table 5.3).

Category	Parameter	Value / Description
Features	Input Representation	Log-Mel Spectrogram
	Augmentation	SpecAugment
Model	Architecture	S-SSSM Encoder + CTC
	Parameter Budget	≈ 8.6 Million (Fixed)
	Layers (L)	$L \in \{9, 12, 18, 24, 30\}$
	Model Dimension (D_{model})	Scaled inversely to L
	Hierarchy (a_0)	$0.15 \rightarrow 0.85$
Training	Optimizer	AdamW
	Learning Rate	1×10^{-3}
	Warmup	500 steps
	Batch Size	32
	Epochs	30

Configurations that include the gating mechanism require a smaller model dimension to compensate for the additional parameters introduced by the doubled input projection. The width adjustment ensures that observed performance differences reflect architectural choices rather than large parameter count disparities. The exact dimensions for each depth configuration are reported in Table 5.3.

4.4.4 Learning Rate Schedule

A cosine learning rate schedule with warmup is used [23]. During the warmup period of $t_{warm} = 500$ steps, the learning rate increases from zero to the peak value η_{max} . After

warmup, the learning rate decays following a cosine curve:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{t_{\text{warm}}} & t < t_{\text{warm}} \\ \frac{\eta_{\max}}{2} \left(1 + \cos \left(\pi \cdot \frac{t - t_{\text{warm}}}{T - t_{\text{warm}}} \right) \right) & t \geq t_{\text{warm}} \end{cases} \quad (4.13)$$

where T is the total number of training steps.

This schedule can aid the optimizer in escaping local minima early in training while also allowing precise convergence in later stages. The warmup period of 500 steps corresponds to approximately half of one epoch on the training set. This duration was chosen to allow the model to see a diverse sample of the training data before reaching the full learning rate, which reduces the risk of early instability on outlier batches.

4.4.5 Mixed Precision Training

To enable training within single-GPU memory constraints, AMP is used [24]. The majority of computations are performed in 16-bit floating point (FP16) to reduce memory usage and increase throughput, while critical operations such as loss computation and weight updates remain in 32-bit (FP32) to maintain numerical accuracy.

4.4.6 Decoding

Inference uses greedy CTC decoding [10], which selects the most probable token at each timestep. The resulting sequence is post-processed by collapsing consecutive repeated tokens and removing the blank symbol to produce the final transcription. While beam search with language model fusion would likely improve absolute performance, greedy decoding is used to isolate the encoder’s contribution without introducing external knowledge sources.

4.4.7 Evaluation Metric

Consistent with the theoretical framework established in Section 2.6.3, model performance is evaluated using WER. As the standard metric for ASR, WER quantifies the Levenshtein distance between the predicted hypothesis and the reference transcription, normalized by sequence length [21]. This metric was chosen to align with standard benchmarks and for comparison with prior work. To quantify the specific performance gains of the proposed architecture, two additional metrics are derived from the WER results.

The *Absolute Improvement* (Δ) measures the percentage point difference in WER rates:

$$\Delta = \text{WER}_{\text{baseline}} - \text{WER}_{\text{new}} \quad (4.14)$$

The *Relative Error Reduction* (RER) normalizes the improvement against the baseline error rate, providing a measure of relative performance difference:

$$\text{RER} = \frac{\text{WER}_{\text{baseline}} - \text{WER}_{\text{new}}}{\text{WER}_{\text{baseline}}} \times 100\% \quad (4.15)$$

4.4.8 Implementation Details

All experiments were conducted on a single NVIDIA V100 GPU. Training used PyTorch [29] with torchaudio [35] for audio processing, and PyTorch Lightning [8] with AMP (FP16) for computational efficiency. Each experimental configuration was trained for 30 epochs with early stopping based on validation WER.

The complete source code, training and evaluation script, and experiment configurations are available on GitHub. Pre-trained model checkpoints are available on Hugging Face.

Related repositories:

- GitHub: [Roberh18/IKT464](#)
- Hugging Face: [Roberh18/IKT464/experiments](#)

4.5 Use of Generative AI and Digital Tools

In accordance with University of Agder guidelines, I declare the following use of generative AI and tools in this work:

ChatGPT: Used for grammar and style editing, refining concepts, debugging code, and clarifying technical explanations [5].

All AI-generated suggestions were critically reviewed and modified to ensure accuracy and alignment with the requirements. The substantive content, including the core implementation, analysis, and conclusions, is entirely my own work.

Overleaf: Used for L^AT_EX writing and formatting.

4.6 Summary of Methodology

This chapter has presented the methodology for investigating hierarchical state dynamics in SSM-based ASR. The proposed system is an encoder-only architecture trained with CTC, designed to isolate the encoder’s contribution to ASR performance.

The processing pipeline transforms raw audio to text through three stages. First, 80-dimensional log-Mel spectrograms are computed and augmented using SpecAugment. Second, a convolutional subsampler reduces the temporal resolution by a factor of four while projecting features into the model dimension. Third, a stack of S-SSSM layers processes the subsampled features before a linear classifier produces character-level predictions.

The core architectural contributions lie in the S-SSSM encoder design. The selective recurrence mechanism uses bounded activations to ensure numerical stability while maintaining input-dependent dynamics. Hierarchical state initialization assigns progressively slower temporal decay to deeper layers, encoding an inductive bias that mirrors the multi-scale structure of human speech processing. An explicit gating mechanism provides complementary within-layer selectivity, allowing the model to suppress uninformative regions while emphasizing critical transitions.

The training procedure utilizes AdamW optimization with cosine learning rate scheduling, mixed precision computation for efficiency, and greedy CTC decoding for evaluation. All experiments maintain a fixed parameter budget of approximately 8.6 million parameters, enabling controlled comparisons across architectural variants. The following chapter presents experimental results evaluating this approach.

Chapter 5

Experiments and Results

This chapter presents the experimental evaluation of the proposed hierarchical SSM encoder. The experiments are designed to address the hypotheses stated in Section 1.3. The evaluation begins by establishing a baseline performance, and then systematically ablate the contributions of hierarchical initialization and gating. For the main experiments, the model was trained on the full `train-clean-100` dataset, validated on the `dev-clean` set, and tested on the `test-clean` and `test-other` sets only during the final evaluation.

5.1 Initial Exploration

Before conducting the full-scale experiments, preliminary hypothesis testing was performed on a small 6,000-sample subset of the training data with 600 validation samples. This smaller scale allowed rapid iteration to verify that the proposed modifications produced meaningful differences before committing to full training runs.

Table 5.1 presents the results of this initial exploration. The baseline S-SSSM model achieved a WER of 71.30% on this small subset. Adding the gating mechanism reduced the WER to 68.98%, a reduction of 2.32%. Adding hierarchical state initialization (without gating) reduced the WER further to 67.56%, a reduction of 3.74% from the baseline.

Table 5.1: Initial exploration on small subset (6k train / 600 val). Performance metrics include WER (%) and Relative Error Reduction (RER) calculated against the Baseline.

Exp	Configuration	Dimensions ($D \times L$)	Params (M)	dev-clean (%)	Abs. (Δ)	RER (%)
01	Baseline (S-SSSM)	192×9	2.16	71.30	reference	reference
02	+ Gating	168×9	2.27	68.98	-2.32 pp	3.25
03	+ Hierarchical states	176×9	2.2	67.56	-3.74 pp	5.25

These preliminary results suggested that hierarchical initialization provided the larger benefit of the two modifications, offering early support for the Hierarchy Hypothesis.

5.2 Architecture Ablation Study

To isolate the contribution of each proposed component and directly test the research hypotheses, a controlled ablation was conducted on the full train-clean-100 dataset. Table 5.2 presents the results of the architecture ablation on both test sets.

All models were trained for 30 epochs with identical hyper-parameters and matched parameter counts of ≈ 8.6 million. The model dimension was adjusted between configurations to maintain the parameter budget. Here, the baseline and hierarchical-only variants used $D = 336$, while the gating variants used $D = 312$ to account for the additional parameters

Table 5.2: Architecture ablation study on LibriSpeech. Performance metrics below include WER (%) and (Abs. Imp. (Δ) / RER), relative to the baseline results.

Exp	Configuration	Dimensions ($D \times L$)	Params (M)	test-clean (%)	test-other (%)
01	Baseline	336×12	8.56	27.78	56.26
02	+ Hierarchical init	336×12	8.56	27.33 (0.45 pp / 1.62%)	55.61 (0.65 pp / 1.16%)
03	+ Gating	312×12	8.56	27.57 (0.21 pp / 0.76%)	56.07 (0.19 pp / 0.34%)
04	+ Hier & Gate	312×12	8.56	27.11 (0.68 pp / 2.43%)	55.23 (1.03 pp / 1.83%)

in the gating pathway. The baseline model achieved a WER of 27.78% on test-clean and 56.26% on test-other. The combination of hierarchical initialization and gating achieved the best performance, with a final WER of 27.11% on test-clean (0.68%, 2.43% RER) and 55.23% on test-other (1.03%, 1.83% RER). Notably, the improvements from the two components were approximately additive, suggesting that they address complementary aspects of the modeling problem.

These results provide direct evidence for evaluating the research hypotheses. The consistent improvement from hierarchical initialization supports the Hierarchy Hypothesis, while the improvement from gating supports the Gating Hypothesis. The additive nature of the improvements suggests that the two mechanisms operate through different pathways.

5.3 Depth vs. Width Analysis

To identify the most parameter-efficient configuration, I investigated the trade-off between model depth (L) and width (D_{model}) under the fixed parameter budget. Specifically, I compared five distinct configurations, ranging from shallow-wide to deep-narrow architectures, to determine the optimal allocation of parameters. Table 5.3 presents the results across different depth-width configurations.

Table 5.3: Impact of dimensionality (Depth vs. Width) on WER, model trained for 30 epochs. Performance metrics below include WER (%) and RER (%) for models across varying depths with fixed parameter budget (≈ 8.6 million), relative to the baseline results.

Depth (L)	Width (D_{model})	Params (M)	test-clean		test-other	
			Baseline WER (%)	Hier & Gate WER & RER (%)	Baseline WER (%)	Hier & Gate WER & RER (%)
9	386 / 358	8.57 / 8.59	29.99	28.98 (-3.4%)	58.67	57.57 (-1.9%)
12	336 / 312	8.56 / 8.56	27.78	27.11 (-2.4%)	56.26	55.23 (-1.8%)
18	276 / 256	8.56 / 8.57	26.25	25.36 (-3.4%)	54.17	53.31 (-1.6%)
24	240 / 222	8.60 / 8.56	25.29	24.66 (-2.5%)	53.46	52.70 (-1.4%)
30	216 / 200	8.68 / 8.66	24.85	24.45 (-1.6%)	53.11	52.42 (-1.3%)
36	196 / 182	8.57 / 8.60	24.51	24.09 (-1.7%)	52.90	52.14 (-1.4%)
42	182 / 168	8.61 / 8.54	24.55	23.93 (-2.5%)	52.47	51.72 (-1.4%)
48	170 / 158	8.58 / 8.63	24.82	23.85 (-3.9%)	52.65	51.86 (-1.5%)
54	160 / 148	8.55 / 8.52	24.77	23.89 (-3.6%)	52.46	51.71 (-1.4%)

Several patterns emerge from these results. First, deeper configurations consistently outperform shallower ones within the tested range. The 30-layer configuration achieves the best

overall performance with a WER of 24.45% on test-clean when using hierarchical initialization and gating, improving upon the 9-layer configuration by over 4.5% absolute. This trend suggests that the additional representational capacity from depth is more beneficial than equivalent capacity from width at this parameter scale.

Second, the benefit of hierarchical initialization and gating is consistent across all depths tested. As illustrated in Figure 5.1, the validation WER curves for the hierarchical models consistently track below the uniform baselines throughout the training process. Each configuration shows improvement when the proposed modifications are added, with relative error reductions ranging from 1.3% to 3.4% on test-clean. This consistency across architectural variations provides support for the generality of the hierarchical initialization principle.

Third, while the 30-layer configuration achieves the highest absolute performance of all models tested (24.45% WER), an interesting pattern emerges in the relative error reduction. While hierarchical initialization provides consistent absolute improvements at all depths, the relative benefit diminishes as depth increases. Configurations $L = 9$ and $L = 18$ both show 3.4% relative improvement, but this diminishes to 1.6% at $L = 30$. This diminishing return suggests that very deep networks may possess greater capacity to discover temporal hierarchies through optimization alone, reducing but not eliminating the benefit of explicit initialization. However, structural constraints likely play a role as well. The reduced width at $L=30$ ($D = 200$) approaches a representational bottleneck. With input features fixed at 80 mel-frequency bins, this configuration has a significantly lower expansion ratio ($\approx 2.5\times$) than wider variants. This narrow latent dimension may be insufficient to fully disentangle complex acoustic features, effectively counteracting the benefits of increased depth and limiting the model’s ability to fully exploit the hierarchical structure.

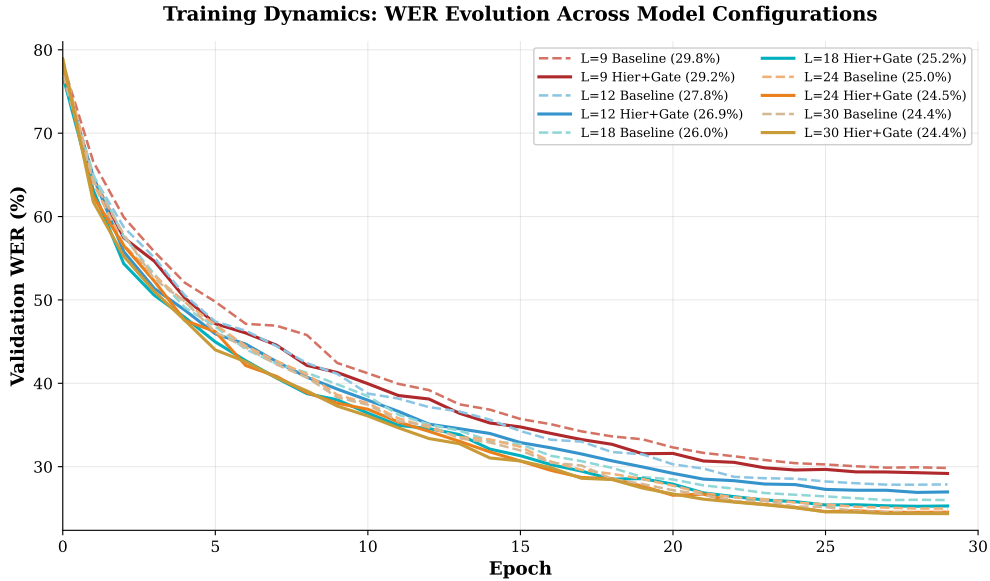


Figure 5.1: Validation WER Evolution Across Model Configurations. Comparison of Baseline (dashed) vs. Hierarchical + Gating (solid) across five model depths ($L = 9$ to $L = 30$). The proposed architecture consistently outperforms the baseline, with deeper models generally achieving lower error rates. The 24-layer and 30-layer Hierarchical+Gating models achieve the best convergence.

5.4 Detailed Analysis

5.4.1 State Dynamics Visualization

To verify whether the hierarchical structure is preserved through training, Figure 5.2 shows the mean learned values of the state dynamics parameters across the 24-layer ($D = 222$)

configuration after training completes. Despite initialization with $a_0 \in [0.15, 0.85]$, the state

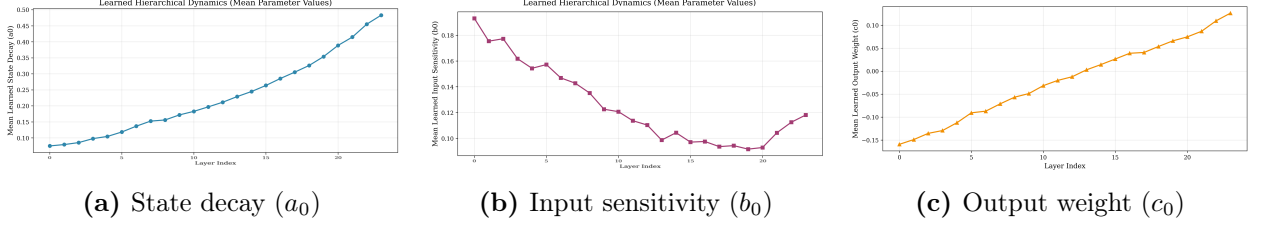


Figure 5.2: Learned hierarchical dynamics parameters for the 24-layer ($D = 222$) configuration. The plots display the mean parameter values for (a) state decay, (b) input sensitivity, and (c) output weight after training.

decay parameter a_0 (Figure 5.2a) converges to values between approximately 0.08 and 0.48. It maintains a monotonically increasing pattern, demonstrating that gradient-based optimization refines rather than destroys the imposed temporal hierarchy. Similarly, the input sensitivity parameter b_0 (Figure 5.2b) exhibits a general decreasing trend from early to late layers, while the output weight c_0 (Figure 5.2c) follows an approximately linear progression.

This behavior is also observed in the deepest configuration. Figure 5.3 presents the dynamics for the 30-layer ($D = 200$) model, explicitly overlaying the *initialized* values (gray) against the final *learned* parameters in (blue). As shown in Figure 5.3, the learned State

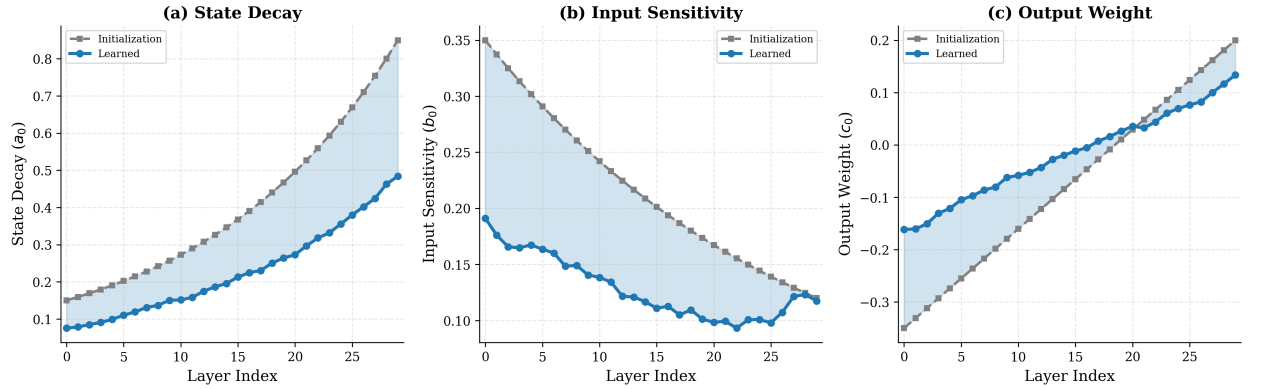


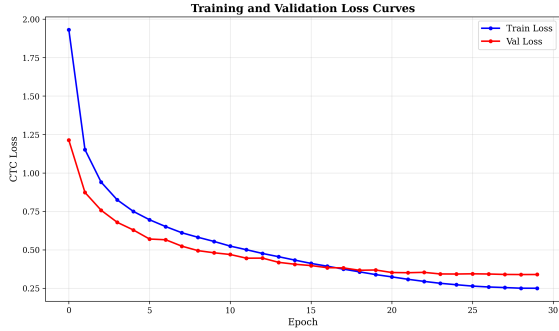
Figure 5.3: Learned hierarchical dynamics parameters for the 30-layer ($D = 200$) configuration. (a) *State Decay* (a_0) retains a monotonic increase, preserving the initialized hierarchy where deeper layers have longer memory. (b) *Input Sensitivity* (b_0) decreases with depth, making deeper layers less sensitive to instantaneous input. (c) *Output Weight* (c_0) maintains a linear progression. The shaded region represents the standard deviation across hidden dimensions.

Decay (a_0) curve strictly tracks the slope of the initialization curve. While the model shifts the absolute values downwards (indicating a preference for slightly faster decay than the heuristic schedule), it preserves the relative hierarchical structure where deep layers retain information significantly longer than early layers. A similar adherence to the initialization structure is observed for Input Sensitivity (b_0) and Output Weight (c_0). These patterns strongly suggest that the hierarchical initialization provides a robust inductive bias. The optimization process evidently finds it beneficial to maintain and fine-tune this structure throughout training, rather than overwrite it.

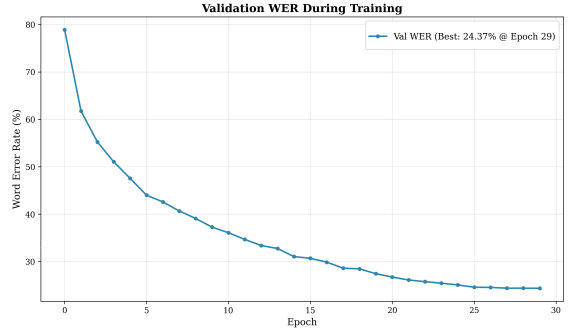
5.4.2 Stability and Convergence

Figure 5.4 shows the training progression for the 30-layer configuration, which is the best-performing one. The CTC loss decreases smoothly for both training and validation sets, with the gap between them remaining small throughout training. The validation WER

decreases steadily from approximately 78% in the first epoch to 24.37% at epoch 29, which tracked closely with the final test-clean WER of 24.45% reported in Table 5.3. The absence of significant loss spikes or WER fluctuations suggests that the combination of hierarchical initialization and gating provides stable training dynamics.



(a) CTC Loss



(b) WER curve

Figure 5.4: Training progression for the 30-layer encoder with hierarchical initialization and gating.

To assess the stability of the reported results, additional experiments were conducted using five different random seeds (42, 123, 456, 707, and 1337) with a fixed configuration of 8 layers, hierarchical initialization, and gating. These experiments used an earlier model dimension of 384, resulting in 8.77 million parameters. Table 5.4 summarizes the results.

The validation WER ranges from 30.68% to 31.51% with a mean of 31.01% and standard deviation of ($\sigma = 0.35\%$). This relatively low variance indicates that the training procedure produces consistent results across different random initializations. The variance provides context for interpreting the improvements reported in the ablation studies. The absolute improvement at $L = 30$ (0.4%) is comparable to the seed variance. This suggests the result may be at or near the margin of statistical significance at this depth configuration, though formal verification would require additional random seed trials at $L = 30$.

Table 5.4: Stability analysis. Validation WER across five random seeds.

Metric	Seed 42	Seed 123	Seed 456	Seed 707	Seed 1337	Mean	Std. Dev.
Val WER (%)	31.51	30.70	30.68	30.90	31.28	31.01	0.35

5.4.3 Comparison with State-of-the-Art

Table 5.5 contextualises the results against published ASR systems. It is important to note that direct comparison of WER values is misleading due to fundamental differences in scale and methodology.

Table 5.5: State-of-the-art ASR results on LibriSpeech.

Model	Encoder	Params	Train Data	test-clean	test-other
<i>State-of-the-Art Systems (960h+ training data)</i>					
Samba-ASR [30]	SSM encoder	$\approx 1B$	15,460h	1.17%	2.48%
Stateformer [9]	Hybrid	140M	960h	1.91%	4.36%
Conformer [14]	Transformer	119M	960h	1.9%	3.9%
<i>This Work (100h training data, CTC-only, greedy decoding)</i>					
Baseline (L=30)	S-SSSM	8.68M	100h	24.85%	53.11%
+ Hier. init & Gating	S-SSSM	8.66M	100h	24.45%	52.42%

State-of-the-art systems such as Samba-ASR achieve WER below 2% on test-clean, compared to 24.45% for the best configuration in this work. However, these systems differ dramatically in scale and methodology, as detailed in Table 5.6. Samba-ASR is a large-scale model (comparable to 1B-parameter benchmarks) trained on 15,460 hours of multi-domain data,

Table 5.6: Training regime comparison.

Model	Training Data	Decoder	LM Fusion	Decoding	Params
Samba-ASR	15,460h (multi-domain)	Mamba (SSM)	Yes	Beam search	$\approx 1B$
Stateformer	960h	Attention	No	Beam search	140M
Conformer	960h	RNN-T	Yes	Beam search	119M
This work	100h	None (CTC only)	No	Greedy	$\approx 8.6M$

utilizing Mamba-based decoding. The model presented here uses ≈ 8.6 million parameters trained on 100 hours with greedy CTC decoding and no external language model. Additionally, the test-other results in this work reflect out-of-distribution generalization proficiency, as the model was trained exclusively on clean speech, whereas state-of-the-art systems train on data that includes acoustically challenging recordings. The relevant comparison is not the absolute WER but rather the improvement achieved by the proposed modifications relative to the baseline under controlled conditions. The configurations with hierarchical initialization and gating consistently achieve better results compared to the uniform baseline. This is accomplished without increasing parameter count or training data, and while modest in absolute terms, demonstrates that explicit temporal inductive biases provide measurable benefit even at small scale. The principle may yield proportionally larger gains when applied to systems with greater capacity and more training data.

5.4.4 Limitations of Analysis

Several limitations should be noted when interpreting these results. First, the experiments were conducted at a single scale (≈ 8.6 million parameters) on a single dataset (LibriSpeech). The benefits of hierarchical initialization may differ at larger or smaller scales, or on datasets with different characteristics.

Second, the hyper-parameter search for the hierarchical initialization schedule was limited. The values for a_{early} , a_{late} , and similar parameters were determined through preliminary experiments rather than systematic optimization. Better choices may exist that would provide larger improvements.

Third, the comparison with state-of-the-art systems is inherently limited by the differences in methodology. The absolute performance gap reflects these differences rather than fundamental limitations of the proposed approach.

5.5 Hypothesis Evaluation

This section provides explicit answers to the hypotheses stated in Section 1.3.

Evaluation of the Hierarchy Hypothesis

The Hierarchy Hypothesis asks whether initializing layers with progressively longer memory spans outperforms standard uniform initialization, and whether this benefit scales with network depth.

Yes, the experimental evidence supports this hypothesis. Across all depth configurations tested, hierarchical initialization improves performance compared to the uniform baseline. In the 12-layer ablation study (Table 5.2), hierarchical initialization alone reduces WER from 27.78% to 27.33% on test-clean, a 1.62% relative error reduction. The pattern holds consistently in the depth versus width experiments (Table 5.3), the 9-layer configuration improves from 29.99% to 28.98%, the 18-layer from 26.25% to 25.36%, and the 24-layer from 25.29% to 24.66% when hierarchical initialization is combined with gating.

The hypothesis that hierarchical benefit increases with network depth receives partial support from the 12-layer architecture ablation study, detailed in Table 5.2. However, it is important to note that the isolated contribution of hierarchical initialization cannot be fully disentangled in the depth analysis (Table 5.3), as these experiments combine hierarchy with gating. While improvements are observed at all depths, the relative error reduction does not monotonically increase with depth. The 9-layer and 18-layer configurations show 3.4% relative improvement, while the 12-layer and 24-layer configurations show 2.4 - 2.5% relative improvement. This variation may reflect interactions between depth, width, and the hierarchical initialization schedule that warrant further investigation.

Evaluation of the Gating Hypothesis

The Gating Hypothesis asks whether the addition of gating mechanisms improves stability, convergence, and accuracy compared to the baseline model.

Yes, the experimental evidence supports this hypothesis, although the isolated impact of gating is more modest than that of hierarchical initialization. In the controlled 12-layer ablation, adding gating alone reduces the WER from 27.78% to 27.57% on test-clean, yielding a 0.76% relative error reduction. More importantly, the combination of gating with hierarchical initialization consistently achieves the best performance at each depth configuration. This additive improvement suggests that the two mechanisms are complementary, with gating likely refining local feature selection within the broader temporal structure established by the hierarchy. This synergy persists across the varying model scales analyzed in the depth versus width experiments (Table 5.3). The combined architecture achieved the lowest WER at every depth tested, with the 30-layer configuration reaching 24.45% on test-clean and 52.42% on test-other.

5.6 Summary of Key Findings

The experimental evaluation supports both hypotheses proposed in this work. Hierarchical state initialization provides consistent improvements over uniform initialization across all model depths tested, with relative error reductions ranging from 1.6% to 3.4%. Gating mechanisms provide improvements that combine approximately additively with hierarchical initialization, suggesting the two techniques address different aspects of the sequence modeling problem.

The 24-layer encoder configuration with hierarchical initialization and gating achieved 24.66% WER on test-clean and 52.70% on test-other. This represents a 2.5% relative error reduction compared to the uniform baseline at matched parameter count. The best results were obtained with the 30-layer encoder configuration with hierarchical initialization and gating, achieving 24.45% WER on test-clean and 52.42% on test-other. While the relative error reduction at this depth (1.6%) is lower than at shallower depths (up to 3.4%), the absolute performance is the best achieved in this study.

The depth-width analysis reveals that deeper configurations consistently outperform shallower ones at a fixed parameter budget, with the 30-layer model achieving the best absolute results. This confirms that allocating capacity to depth rather than width is beneficial at this parameter scale (≈ 8.6 million). However, the diminishing RER at greater depths (1.6% at $L=30$ versus 2.5% at $L=24$) suggests that the benefit of hierarchical initialization may plateau or that reduced width limits the ability to fully exploit the temporal hierarchy.

These findings suggest that explicit temporal inductive biases, encoded through layer-wise initialization of state dynamics, provide measurable benefit for SSM-based ASR.

Chapter 6

Discussion

This chapter interprets the experimental findings, examines their implications for SSM-based speech recognition, and addresses the limitations of the current work.

6.1 Interpretation of Results

The consistent improvement from hierarchical initialization across all tested configurations suggests that the temporal structure of speech is not fully discovered through gradient-based optimization alone. By providing an initialization that matches the expected timescale hierarchy, the model begins training with a structure that appears to facilitate learning. Early layers initialized for fast dynamics can immediately begin capturing acoustic transients, while deep layers initialized for slow dynamics can integrate information over longer spans without first having to learn this behavior from scratch.

The complementary benefit of gating supports the hypothesis that speech contains variable information density. The gating mechanism allows the model to suppress uninformative regions such as steady-state vowels or silence while emphasizing transitions and boundaries. That gating and hierarchical initialization provide approximately additive improvements suggests they operate through different mechanisms. The hierarchical initialization structures what timescales each layer attends to, while gating controls which timesteps receive emphasis within each layer.

The observation that deeper models outperform shallower ones at fixed parameter count has implications for architecture design. At the 8.6 million parameter scale explored in this work, allocating capacity to depth rather than width appears beneficial, at least up to 30 layers. This may reflect the sequential nature of speech processing, where building representations through multiple stages of transformation is more effective than processing with fewer but wider layers.

The preservation of hierarchical structure after training has implications for understanding how the model processes speech. If gradient descent consistently moved the parameters toward uniform values across layers, this would suggest the hierarchy provides no lasting benefit beyond potentially faster initial convergence. Instead, the maintained differentiation suggests that different layers genuinely benefit from different temporal dynamics. Early layers with fast decay can specialize in capturing transient acoustic events that occur on short timescales, while deep layers with slower decay can integrate information over the longer spans needed for linguistic processing. The fact that the final a_0 values (0.08 to 0.48) differ from the initialization (0.15 to 0.85) indicates that the model does adapt the dynamics during training, but it adapts within the hierarchical framework rather than abandoning it.

6.2 Comparison with Alternative Approaches

The hierarchical initialization strategy investigated in this work differs from prior approaches in its simplicity and efficiency. Multi-head SSM architectures such as Stateformer process multiple timescales in parallel, requiring proportionally more computation. Chunked hierarchical models such as HiSS introduce explicit multi-resolution processing but create boundary artifacts and lose the continuous-time advantages of SSMs. The layer-wise initialization approach maintains a standard encoder architecture with no additional computational overhead while still encoding temporal hierarchy.

The improvements achieved are modest compared to the gains from scaling to more parameters or training data. However, the approach is orthogonal to scaling and could potentially be combined with larger models. Unlike gating or multi-head mechanisms which require parameter trade-offs, this initialization strategy costs nothing in terms of parameters or computation, making it essentially free to incorporate into any SSM-based system.

6.3 Limitations

Several limitations constrain the conclusions that can be drawn from this work. First, all experiments were conducted at a single parameter scale of approximately 8.6 million parameters. The benefits of hierarchical initialization may differ at larger scales where models have greater capacity to discover temporal structure through optimization alone. Conversely, the benefits might increase at larger scales if the inductive bias helps optimization navigate a more complex loss landscape.

Second, the experiments used only the LibriSpeech corpus, which consists of read speech in controlled acoustic conditions. The benefits of hierarchical initialization for spontaneous speech, accented speech, or noisy recordings remain unknown. The temporal structure of these speech types may differ from read speech in ways that affect the optimal initialization strategy.

Third, the specific initialization values were determined through limited preliminary experiments rather than systematic optimization. The range $[0.15, 0.85]$ for the decay parameter and the exponential interpolation schedule were selected based on reasoning about activation function behavior and informal experimentation. A more thorough hyper-parameter search might identify better values.

Finally, the use of CTC-only decoding, while useful for isolating encoder contributions, limits the absolute performance achievable. State-of-the-art systems use attention-based decoders, language model fusion, and beam search decoding. Whether hierarchical initialization provides similar benefits when combined with these techniques is an open question.

Chapter 7

Conclusions

This work investigated whether explicit temporal inductive biases can improve SSM-based ASR. The central hypothesis was that initializing SSM layers with progressively slower temporal dynamics, matching the hierarchical timescale structure of human speech processing, would outperform standard uniform initialization.

The experimental evaluation supports this hypothesis. A S-SSSM encoder was implemented with layer-wise initialization of state decay parameters, where early layers were configured for fast dynamics suitable for acoustic processing and deep layers for slow dynamics suitable for linguistic integration. Systematic ablation studies on the LibriSpeech 100-hour corpus demonstrated that hierarchical initialization consistently improves performance across model depths ranging from 9 to 30 layers.

The best configuration, a 30-layer encoder with hierarchical initialization and gating, achieved 24.45% WER on test-clean and 52.42% on test-other. The relative error reduction at this depth (1.6%) is smaller than at moderate depths (2.5 – 3.4%), indicating diminishing returns of hierarchical initialization as network depth increases, though the absolute improvement persists.

While the absolute performance remains far from state-of-the-art systems using more parameters and training data, the architectural principle of hierarchical state dynamics costs nothing in terms of parameters or computation. The improvement is achieved purely through initialization, making it readily applicable to any SSM-based speech recognition system.

The findings contribute to the understanding of how inductive biases can improve sequence models for speech. Rather than relying entirely on optimization to discover temporal structure, explicitly encoding this structure into the initialization provides a head start that translates to measurable performance gains. This principle may extend beyond speech recognition to other domains where signals exhibit hierarchical temporal structure.

Chapter 8

Future Work

Several directions emerge from this work that warrant further investigation. One direction would be to evaluate hierarchical initialization at larger scales. The experiments were constrained to approximately 8.6 million parameters, which forced a trade-off between depth and width. Testing with models in the 15-30 million parameter range would allow for testing of deep networks that maintain sufficient width to determine whether the diminishing returns observed at $L = 30$ were caused by the initialization schedule or the representational bottleneck of narrow layers. Additionally, training the model on 460-hours, or the full 960-hour LibriSpeech corpus would reveal whether the benefits of hierarchical initialization persist, diminish, or potentially increase at scale.

A second direction is to combine hierarchical initialization with more sophisticated decoding strategies. This work used greedy CTC decoding to isolate encoder contributions, but practical systems use beam search with language model fusion. Investigating whether hierarchical initialization provides similar benefits in hybrid CTC-attention architectures or with external language models would establish the practical relevance of the approach.

A third direction is to optimize the initialization schedule itself. The current work used fixed values for the initialization range and interpolation scheme based on limited preliminary experiments. Treating these as hyper-parameters to be optimized, potentially through methods such as Bayesian optimization, might identify configurations that provide larger improvements. Alternatively, learning the initialization schedule as part of a meta-learning procedure could adapt the hierarchy to specific datasets or domains.

A fourth direction is investigating depth-adaptive initialization schedules. The diminishing relative benefit of hierarchical initialization at $L = 30$ suggests that the fixed schedule ($a_0 \in [0.15, 0.85]$) may not be optimal for all depths. Future work could explore initialization ranges that vary dynamically with encoder depth, potentially recovering larger relative improvements in very deep networks where the signal propagation properties differ from shallower ones.

A fifth direction is to extend the approach to other sequence modeling domains. The hierarchical temporal structure that motivates this work is not unique to speech. Music, video, and physiological signals such as electrocardiograms exhibit similar multi-scale structure. Adapting the initialization strategy to these domains and evaluating its effectiveness would establish the generality of the principle beyond speech recognition.

Finally, investigating bidirectional variants of the hierarchical encoder could address a limitation of the current unidirectional architecture. Speech recognition benefits from future context, and bidirectional models typically outperform unidirectional ones. However, extending hierarchical initialization to this setting presents a unique challenge, particularly if the forward and backward passes require different temporal dynamics to model the asymmetry of speech perception.

Bibliography

- [1] Y. Bengio, P. Simard, and P. Frasconi. “Learning long-term dependencies with gradient descent is difficult.” In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166. DOI: [10.1109/72.279181](https://doi.org/10.1109/72.279181).
- [2] Raunaq Bhirangi et al. “HiSS: Hierarchical State Space Models for Continuous Sequence-to-Sequence Modeling.” In: *arXiv preprint arXiv:2402.10211* (2024).
- [3] Charlotte Caucheteux, Alexandre Gramfort, and Jean-Remi King. *Long-range and hierarchical language predictions in brains and algorithms*. 2021. arXiv: [2111.14232](https://arxiv.org/abs/2111.14232) [q-bio.NC]. URL: <https://arxiv.org/abs/2111.14232>.
- [4] William Chan et al. “Listen, attend and spell: A neural network for large vocabulary conversational speech recognition.” In: *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2016, pp. 4960–4964. DOI: [10.1109/ICASSP.2016.7472621](https://doi.org/10.1109/ICASSP.2016.7472621).
- [5] *ChatGPT*. [Online; accessed 2025-11-02]. URL: <https://chatgpt.com/>.
- [6] Guoguo Chen et al. “GigaSpeech: An Evolving, Multi-Domain ASR Corpus with 10,000 Hours of Transcribed Audio.” In: *Interspeech 2021*. ISCA, Aug. 2021. DOI: [10.21437/interspeech.2021-1965](https://doi.org/10.21437/interspeech.2021-1965). URL: <http://dx.doi.org/10.21437/Interspeech.2021-1965>.
- [7] Junyoung Chung et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. arXiv: [1412.3555](https://arxiv.org/abs/1412.3555) [cs.NE]. URL: <https://arxiv.org/abs/1412.3555>.
- [8] William Falcon and The PyTorch Lightning team. *PyTorch Lightning*. 2019. URL: <https://github.com/Lightning-AI/lightning>.
- [9] Yassir Fathullah et al. *Multi-Head State Space Model for Speech Recognition*. 2023. arXiv: [2305.12498](https://arxiv.org/abs/2305.12498) [eess.AS]. URL: <https://arxiv.org/abs/2305.12498>.
- [10] Alex Graves et al. “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks.” In: vol. 2006. Jan. 2006, pp. 369–376. DOI: [10.1145/1143844.1143891](https://doi.org/10.1145/1143844.1143891).
- [11] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024. arXiv: [2312.00752](https://arxiv.org/abs/2312.00752) [cs.LG]. URL: <https://arxiv.org/abs/2312.00752>.
- [12] Albert Gu, Karan Goel, and Christopher Ré. *Efficiently Modeling Long Sequences with Structured State Spaces*. 2022. arXiv: [2111.00396](https://arxiv.org/abs/2111.00396) [cs.LG]. URL: <https://arxiv.org/abs/2111.00396>.
- [13] Albert Gu et al. *On the Parameterization and Initialization of Diagonal State Space Models*. 2022. arXiv: [2206.11893](https://arxiv.org/abs/2206.11893) [cs.LG]. URL: <https://arxiv.org/abs/2206.11893>.
- [14] Anmol Gulati et al. *Conformer: Convolution-augmented Transformer for Speech Recognition*. 2020. arXiv: [2005.08100](https://arxiv.org/abs/2005.08100) [eess.AS]. URL: <https://arxiv.org/abs/2005.08100>.
- [15] Uri Hasson. “Uncovering a Timescale Hierarchy by Studying the Brain in a Natural Context.” eng. In: *The Journal of neuroscience : the official journal of the Society for Neuroscience* 45 (Mar. 2025). ISSN: 1529-2401. DOI: [10.1523/JNEUROSCI.2368-24.2025](https://doi.org/10.1523/JNEUROSCI.2368-24.2025).
- [16] Gregory Hickok and David Poeppel. “The cortical organization of speech processing.” In: *Nature Reviews Neuroscience* 8.5 (2007), pp. 393–402.
- [17] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory.” In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).

- [18] I. M. Dushyanthi Karunathilake et al. “Neural Dynamics of the Processing of Speech Features: Evidence for a Progression of Features from Acoustic to Sentential Processing.” In: *Journal of Neuroscience* 45.11 (2025). ISSN: 0270-6474. DOI: [10.1523/JNEUROSCI.1143-24.2025](https://doi.org/10.1523/JNEUROSCI.1143-24.2025). eprint: <https://www.jneurosci.org/content/45/11/e1143242025.full.pdf>. URL: <https://www.jneurosci.org/content/45/11/e1143242025>.
- [19] Suyoun Kim, Takaaki Hori, and Shinji Watanabe. *Joint CTC-Attention based End-to-End Speech Recognition using Multi-task Learning*. 2017. arXiv: [1609.06773](https://arxiv.org/abs/1609.06773) [cs.CL]. URL: <https://arxiv.org/abs/1609.06773>.
- [20] Duc Le et al. *Deep Shallow Fusion for RNN-T Personalization*. 2020. arXiv: [2011.07754](https://arxiv.org/abs/2011.07754) [cs.CL]. URL: <https://arxiv.org/abs/2011.07754>.
- [21] Vladimir I. Levenshtein. “Binary codes capable of correcting deletions, insertions, and reversals.” In: *Soviet physics. Doklady* 10 (1965), pp. 707–710. URL: <https://api.semanticscholar.org/CorpusID:60827152>.
- [22] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. 2019. arXiv: [1711.05101](https://arxiv.org/abs/1711.05101) [cs.LG]. URL: <https://arxiv.org/abs/1711.05101>.
- [23] Ilya Loshchilov and Frank Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts*. 2017. arXiv: [1608.03983](https://arxiv.org/abs/1608.03983) [cs.LG]. URL: <https://arxiv.org/abs/1608.03983>.
- [24] Paulius Micikevicius et al. *Mixed Precision Training*. 2018. arXiv: [1710.03740](https://arxiv.org/abs/1710.03740) [cs.AI]. URL: <https://arxiv.org/abs/1710.03740>.
- [25] Takafumi Moriya et al. “Attention-Free Dual-Mode ASR with Latency-Controlled Selective State Spaces.” In: Aug. 2025, pp. 3588–3592. DOI: [10.21437/Interspeech.2025-1079](https://doi.org/10.21437/Interspeech.2025-1079).
- [26] OpenSLR and Hugging Face. *LibriSpeech ASR Dataset*. Hugging Face Hub. Accessed: November 13, 2025. 2020. URL: https://huggingface.co/datasets/openslr/librispeech_asr.
- [27] Vassil Panayotov et al. “Librispeech: an ASR corpus based on public domain audio books.” In: *Acoustics, Speech, and Signal Processing (ICASSP), 2015 IEEE International Conference on*. IEEE. 2015, pp. 5206–5210.
- [28] Daniel S. Park et al. “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition.” In: *Interspeech 2019*. ISCA, Sept. 2019, pp. 2613–2617. DOI: [10.21437/interspeech.2019-2680](https://doi.org/10.21437/interspeech.2019-2680). URL: <http://dx.doi.org/10.21437/Interspeech.2019-2680>.
- [29] Adam Paszke, Sam Gross, Francisco Massa, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. URL: <https://pytorch.org/>.
- [30] Syed Abdul Gaffar Shakhadri, Kruthika KR, and Kartik Basavaraj Angadi. *Samba-ASR: State-Of-The-Art Speech Recognition Leveraging Structured State-Space Models*. 2025. arXiv: [2501.02832](https://arxiv.org/abs/2501.02832) [cs.CL]. URL: <https://arxiv.org/abs/2501.02832>.
- [31] Jimmy T. H. Smith, Andrew Warrington, and Scott W. Linderman. *Simplified State Space Layers for Sequence Modeling*. 2023. arXiv: [2208.04933](https://arxiv.org/abs/2208.04933) [cs.LG]. URL: <https://arxiv.org/abs/2208.04933>.
- [32] Shriyank Somvanshi et al. *From S4 to Mamba: A Comprehensive Survey on Structured State Space Models*. 2025. arXiv: [2503.18970](https://arxiv.org/abs/2503.18970) [cs.LG]. URL: <https://arxiv.org/abs/2503.18970>.
- [33] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [34] Shinji Watanabe et al. “Hybrid CTC/Attention Architecture for End-to-End Speech Recognition.” In: *IEEE Journal of Selected Topics in Signal Processing* 11.8 (2017), pp. 1240–1253. DOI: [10.1109/JSTSP.2017.2763455](https://doi.org/10.1109/JSTSP.2017.2763455).
- [35] Yao-Yuan Yang et al. “TorchAudio: Building Blocks for Audio and Speech Processing.” In: *IEEE International Conference on Acoustics, Speech and Signal Processing*. 2022.